

COMPUTER SCIENCE SENIOR DESIGN PROJECT REPORT

WHISPERWIND GROVE

An AI-Enhanced Mini-Game Platform with Persistent Player Progression

Written by:
Ayse Sule Ekiz

Faculty Advisor:
Glory Kurian

Department of Computer Science

North American University

Spring 2026

Repository: github.com/aysesule2405/WW

Abstract

Whisperwind Grove is a full-stack browser game platform that combines four short-form mini-games with persistent player identity, score history, leaderboards, achievements, AI-assisted interaction, and character voice synthesis. The platform is implemented as a TypeScript monorepo with a Vite 7 and React 19 frontend, a Node.js and Express 5 backend, Phaser 3.90 game scenes, and MongoDB persistence through Mongoose 9.

The platform contains four playable worlds. Spirit Drift is an arcade catching game built with Phaser 3. Delivery on the Wind is a timed package-delivery game with pathing, inspection modals, and fastest-time persistence. Spirit Sapling is a nurturing game where player kindness is evaluated through Gemini 2.0 Flash and converted into faster energy recharge. Rise of the Half Moon is a three-level moon-phase card-placement strategy game with local AI and optional Gemini-assisted move selection.

The core engineering contribution is the integration of game-specific state machines with a shared persistence layer. Each completed game submits session and score data to a MongoDB-backed API. The backend evaluates achievements using idempotent Badge and UserBadge records with MongoDB upsert semantics, updates UserHighScore documents, and returns achievement payloads that the frontend displays through animated unlock toasts. AI capabilities are intentionally bounded: Gemini 2.0 Flash evaluates short kindness messages and suggests legal Half Moon moves through a backend proxy, while ElevenLabs generates guardian voice clips using server-side API keys and per-guardian voice settings.

The frontend uses CSS variables and React style objects for theming rather than a utility CSS framework, and the application uses AppShell state-based navigation rather than URL-based routing. All backend API calls use the native Fetch API. The active persistence layer is MongoDB only; no MySQL, Prisma, or PostgreSQL dependencies are present in the current codebase.

Table of Contents

1. Introduction	4
2. Problem Statement	6
3. Project Goals	7
4. Technology Stack	9
5. Monorepo Structure	10
6. Frontend Architecture	11
6.1 Root Application Flow	
6.2 Dashboard Shell	
6.3 Shared API Layer	
6.4 Per-Game Color Palette System	
7. Phaser Integration	15
7.1 Scene Configuration	
7.2 React/Phaser Boundary	
7.3 Games Using Phaser	
8. Backend Architecture	16
8.1 API Reference	
9. Database Design	18
9.1 Collections	
9.2 Flexible Session Schema	
9.3 Indexing Strategy	
9.4 Score and Session Synchronization	
10. Achievement System	23
10.1 Badge Definitions	
10.2 Idempotent Unlocking	
10.3 Catalog Fallback	
10.4 Reconciliation from Existing History	
10.5 Leaderboard	
11. Game Design and Mechanics	26
11.1 Spirit Drift	
11.2 Spirit Sapling	
11.3 Delivery on the Wind	
11.4 Rise of the Half Moon	
12. AI Integration	35
12.1 Gemini Kindness Evaluation — Spirit Sapling	
12.2 Gemini Move Proxy — Rise of the Half Moon	
12.3 ElevenLabs Guardian Voice	
13. Security and Configuration	38
14. Testing and Verification	39
15. Engineering Challenges and Solutions	40
15.1 Keeping MongoDB as the Only Persistence Path	
15.2 Score and Session Synchronization	
15.3 AI Output Reliability	
15.4 Achievement Recovery	
15.5 Spirit Sapling Layout Responsiveness	
15.6 Phaser and React Lifecycle Synchronization	
16. Future Improvements	42
17. Conclusion	45
References	46

1. Introduction

The modern web has evolved into a rich, interactive canvas capable of sustaining deep narrative experiences and emotionally engaging gameplay loops. Yet browser-based interactive experiences consistently underdeliver on depth, personalization, and engagement: most web game platforms are architecturally stateless, lack persistent player identity, and treat AI as a back-end utility rather than as a first-class gameplay mechanic.

Whisperwind Grove was designed as a coherent answer to these limitations. It is a place made for quiet moments, where each game is short, calm, and complete - sharing a single authentication system, progress tracker, leaderboard, and achievement framework. The platform name and identity reflect this philosophy: a grove of four distinct worlds, each adding a different dimension to the player's permanent record on the platform.

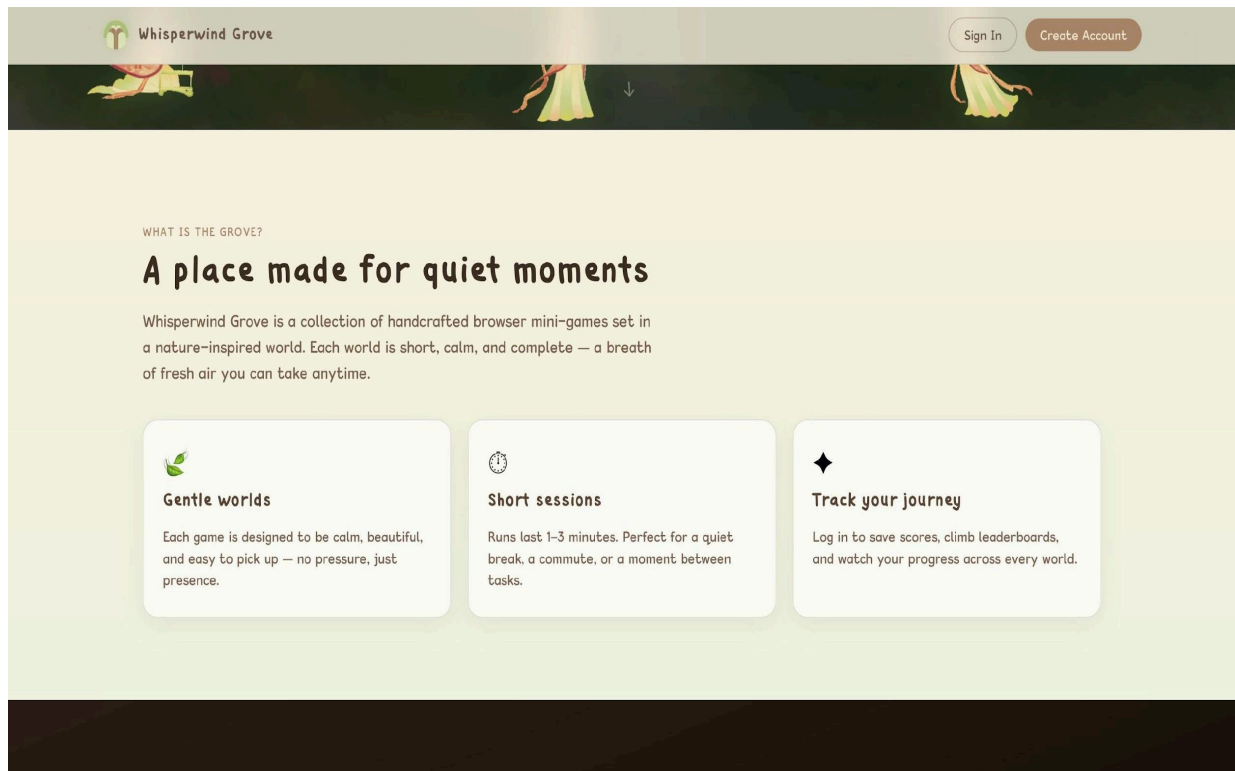


Figure 1.1 - The Whisperwind Grove public landing page. The platform communicates its three core value propositions - Gentle worlds, Short sessions, and Track your journey - before the player has authenticated, establishing the expectation of persistent identity from first contact.

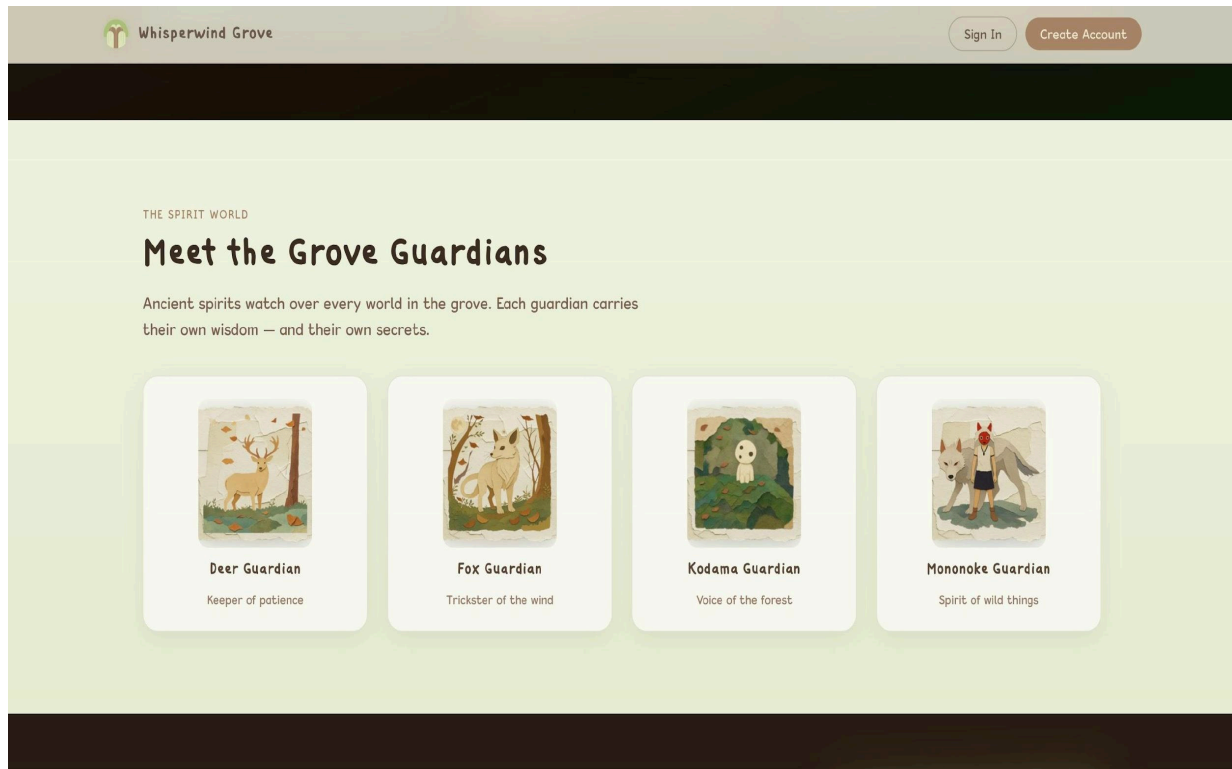


Figure 1.2 - The Grove Guardians section, introducing Deer, Fox, Kodama, and Mononoke - the four nature spirits central to Spirit Sapling and voiced through ElevenLabs TTS with per-guardian voice IDs stored securely in the backend environment.

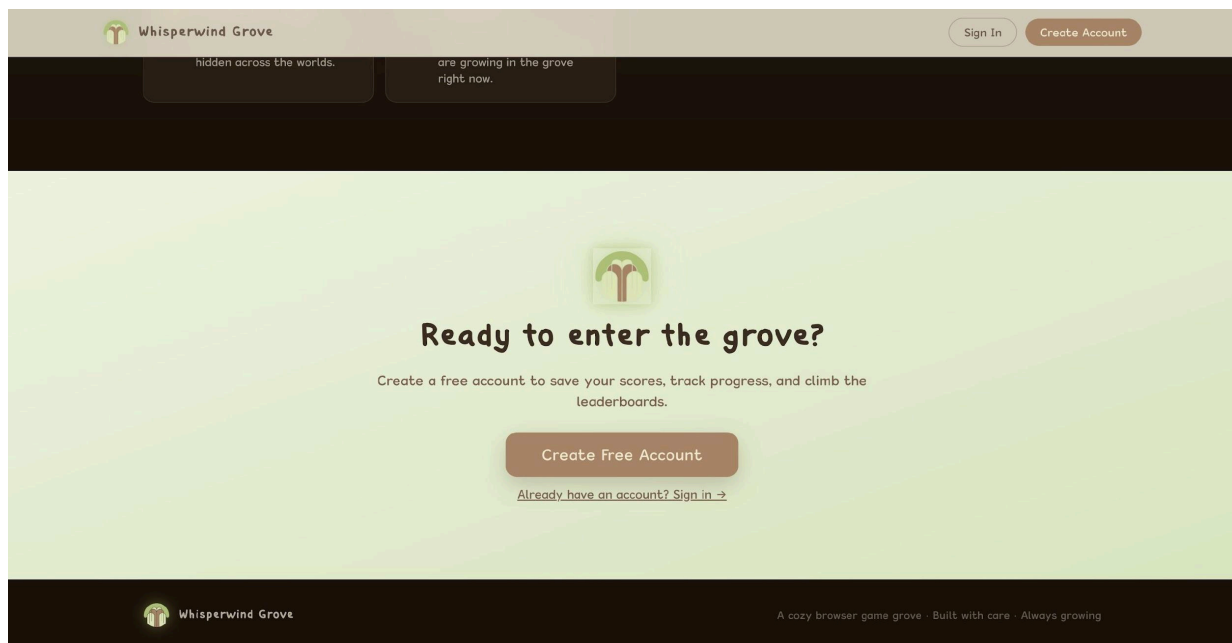


Figure 1.3 - The registration call-to-action and footer. Inviting players to save scores, track progress, and climb leaderboards communicates the platform's persistent identity model before signup.

2. Problem Statement

Browser-based game experiences consistently fail to provide the kind of persistent identity that gives a player a reason to return. Three structural problems are most acute. First, most web game platforms are architecturally stateless: scores vanish on page refresh, there is no cross-session progress, and no social frame, such as a leaderboard, to contextualize individual performance. Second, when AI features appear in browser games, they are typically cosmetic - a chatbot on a loading screen rather than a mechanic that modifies game state. Third, because each game on a typical platform exists in isolation, there is no ecosystem effect: completing one game earns nothing toward any other goal.

Whisperwind Grove addresses all three problems directly. A MongoDB persistence layer stores every score, session, and achievement permanently. Gemini 2.0 Flash is used as a genuine game mechanic - evaluating player kindness to modify Spirit Sapling's recharge timer, and proposing strategic moves in Rise of the Half Moon. A cross-game achievement and leaderboard system ensures that every game contributes to a single unified player profile, creating the ecosystem effect that isolated platforms cannot offer.

3. Project Goals

Whisperwind Grove was designed around four primary goals, each of which shaped concrete architectural and design decisions.

The first goal was to create a cohesive mini-game platform rather than isolated games. All four games share authentication, player profile data, progress tracking, leaderboards, achievements, audio behavior, and dashboard navigation. No game can exist without the platform, and the platform is meaningless without the games.

The second goal was to preserve meaningful player history. Scores, sessions, high scores, completion times, achievement unlocks, and game-specific metadata are stored in MongoDB, so progress survives browser refreshes and future visits. Three separate MongoDB models - ScoreSubmission, GameSession, and UserHighScore - capture different granularities of player history and serve different query patterns.

The third goal was to use AI as a gameplay mechanic, not only as a content generator. In Spirit Sapling, approved kind messages reduce energy recharge time by up to 8 seconds through a bounded growthBoost parameter returned by the Gemini API. In Rise of the Half Moon, Gemini can propose strategic moves, but the client validates every move before accepting it, ensuring the game's logical integrity is never at risk from malformed model output.

The fourth goal was to keep the experience emotionally coherent. The interface uses gentle color palettes, storybook assets, guardian characters with distinct ElevenLabs voices, animated feedback, and calm pacing to make the platform feel like one world.

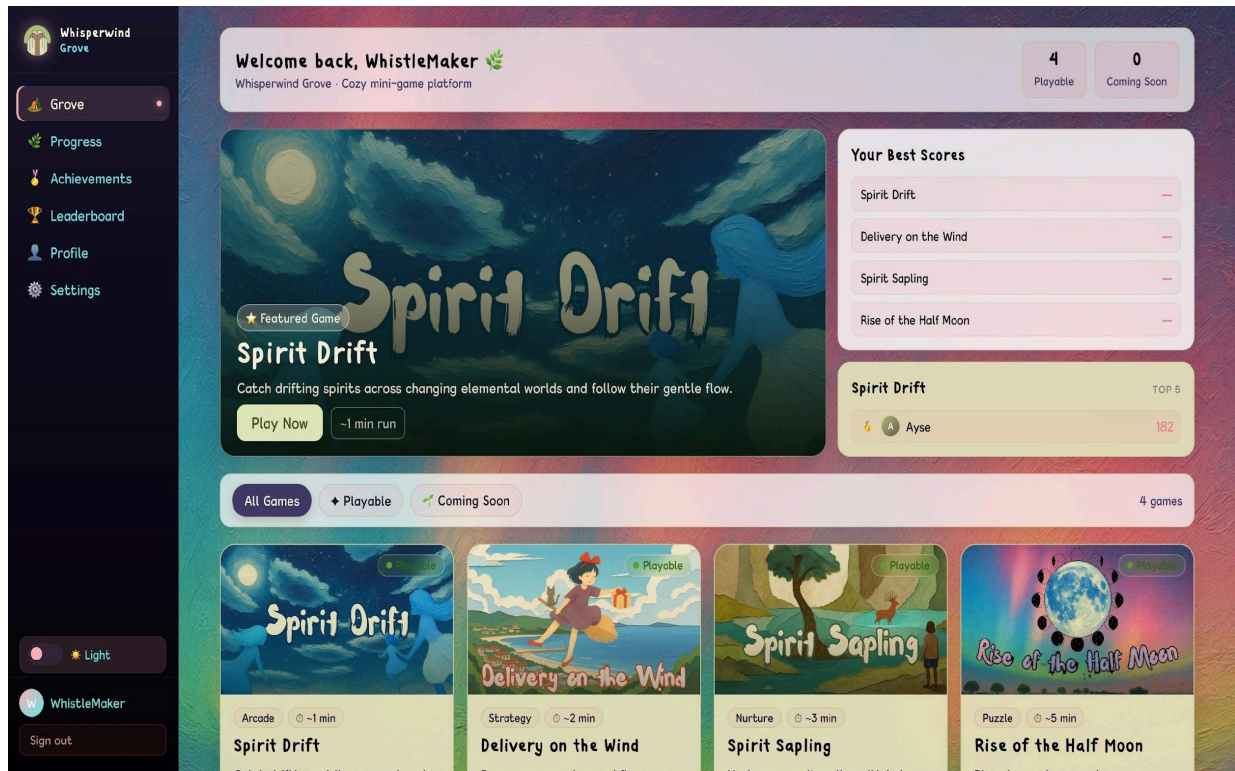


Figure 3.1 - The authenticated dashboard, showing the full platform ecosystem: sidebar navigation (Grove, Progress, Achievements, Leaderboard, Profile, Settings), the featured Spirit Drift banner, all four game cards with playable status, and the cross-game Best Scores panel. Navigation is managed entirely through AppShell React state rather than URL-based routing.

4. Technology Stack

The following table documents the current technology stack as implemented in the repository. Several corrections from the original report are noted in this section.

Layer	Technology	Role
Frontend framework	React 19 + TypeScript	UI composition, game wrappers, dashboard pages, and auth state
Build tool	Vite 7	Local development server and production bundling
Game engine	Phaser 3.90	Canvas/WebGL rendering for Spirit Drift, Delivery on the Wind, and Rise of the Half Moon
Styling	CSS variables, index.css, React style objects	Theming, layout, animations, per-game screens - no Tailwind dependency
Backend	Node.js + Express 5	REST API, auth, score/session persistence, AI proxies
Database	MongoDB	Active and only persistence layer
ODM	Mongoose 9	Schema definitions, indexes, model validation, queries
Auth	JWT + bcrypt	Login/register, signed bearer tokens, password hashing
AI language model	Google Gemini 2.0 Flash	Spirit Sapling kindness evaluation and Half Moon AI move proxy
Voice synthesis	ElevenLabs TTS	Guardian voice playback with per-guardian voice IDs
HTTP client	Fetch API (native)	All frontend API calls - no axios dependency
Security/middleware	helmet, cors, dotenv	HTTP headers, cross-origin access, and environment configuration
Verification	TypeScript, ESLint, Vite build	Static checks and production build validation

5. Monorepo Structure

The project is organized as a monorepo with a clear ownership boundary: the frontend owns presentation, game mounting, local UI state, and API calls; the backend owns persistence, authentication, protected routes, leaderboard queries, achievement evaluation, and all external AI and TTS secrets.

```
ww/
  frontend/
    src/
      components/ # Shared UI components including AppShell, Sidebar,
AchievementToast
      games/      # Phaser game wrappers (SpiritDriftGame, DeliveryOnTheWindGame,
etc.)
      game/       # Phaser scene configurations and createGame factory
      pages/      # Dashboard page components (ProgressPage, LeaderboardPage,
etc.)
      context/    # AuthContext providing user state and token management
      lib/        # api.ts fetch helper, token utilities
      theme/      # Per-game CSS variable definitions
      package.json
      vite.config.ts

  backend/
    src/
      app/        # Express app factory, middleware registration
      config/     # Database connection, environment loading
      core/       # authMiddleware, error handling
      models/     # Mongoose models: User, Game, GameSession, ScoreSubmission,
                  #   UserHighScore, Badge, UserBadge
      modules/    # Feature modules: auth, users, scores, sessions,
                  #   achievements, tts, spiritSapling, halfMoon
      routes/     # Central router composing all module routes
      scripts/    # Seed and migration utilities
      package.json
```

6. Frontend Architecture

The frontend is a Vite 7 single-page React 19 application. It does not use URL-based routing for the authenticated dashboard; instead, AppShell stores the active sidebar section in React state and conditionally renders the selected page. This is a pragmatic design for a capstone game dashboard because it keeps navigation lightweight and avoids unnecessary route-level complexity.

6.1 Root Application Flow

The root App component separates unauthenticated and authenticated surfaces through a simple conditional render. When a user is authenticated and has selected a game, that game replaces the entire shell with its full-screen experience. Exiting the game returns to the dashboard shell.

```
// frontend/src/App.tsx
if (user) {
  const exitGame = () => setActiveGame(null)

  if (activeGame === 'spirit-drift')
    return <SpiritDriftGame onExit={exitGame} />
  if (activeGame === 'delivery-on-the-wind')
    return <DeliveryOnTheWindGame onExit={exitGame} />
  if (activeGame === 'spirit-sapling')
    return <SpiritSaplingGame onExit={exitGame} />
  if (activeGame === 'half-moon')
    return <HalfMoonGame onExit={exitGame} />

  return <AppShell onSelect={ (id) => setActiveGame(id) } onLogout={logout} />
}
```

6.2 Dashboard Shell

The AppShell component renders a persistent Sidebar alongside a main content region. It manages the selected dashboard section in local state, switching between GameSelectionScreen, ProgressPage, AchievementsPage, LeaderboardPage, ProfilePage, and SettingsPage. This design keeps game state fully separate from dashboard state: when a Phaser game unmounts, its instance can be destroyed without affecting any dashboard page.

```
// frontend/src/components/layout/AppShell.tsx
const [section, setSection] = useState<SidebarSection>('games')

<Sidebar active={section} onChange={setSection}
  username={user?.username ?? ''} />
<main>
  {section === 'games'      && <GameSelectionScreen onSelect={onSelect} />}
  {section === 'progress'   && <ProgressPage />}

```

```
{section === 'achievements' && <AchievementsPage />}
{section === 'leaderboard' && <LeaderboardPage />}
{section === 'profile' && <ProfilePage />}
{section === 'settings' && <SettingsPage />}
</main>
```

6.3 Shared API Layer

All frontend HTTP calls use a shared fetch-based helper in `src/lib/api.ts`. The module exports an `apiUrl` function that constructs endpoint paths relative to the `VITE_API_BASE` environment variable, and an `authHeaders` function that attaches the JWT bearer token from local storage. This centralizes deployment configuration so that the API base can be changed without modifying individual components.

```
// frontend/src/lib/api.ts
export const API_BASE = import.meta.env.VITE_API_BASE ?? '/api/v1.'

export function apiUrl(path: string): string {
  return `${API_BASE}${path.startsWith('/') ? path : `/${path}`}`
}

function authHeaders(): Record<string, string> {
  const token = getToken()
  return token ? { Authorization: `Bearer ${token}` } : {}
}
```

6.4 Per-Game Color Palette System

Each game ships with a dedicated light and dark color palette, implemented as CSS variables in the theme directory and applied as React style objects within game components. The Dashboard uses a forest green and warm parchment palette, Spirit Drift uses deep navy and sky blue, and Delivery on the Wind uses warm terracotta and earth tones. These palettes are not managed by a utility CSS framework but through direct CSS variable assignment at the component level.

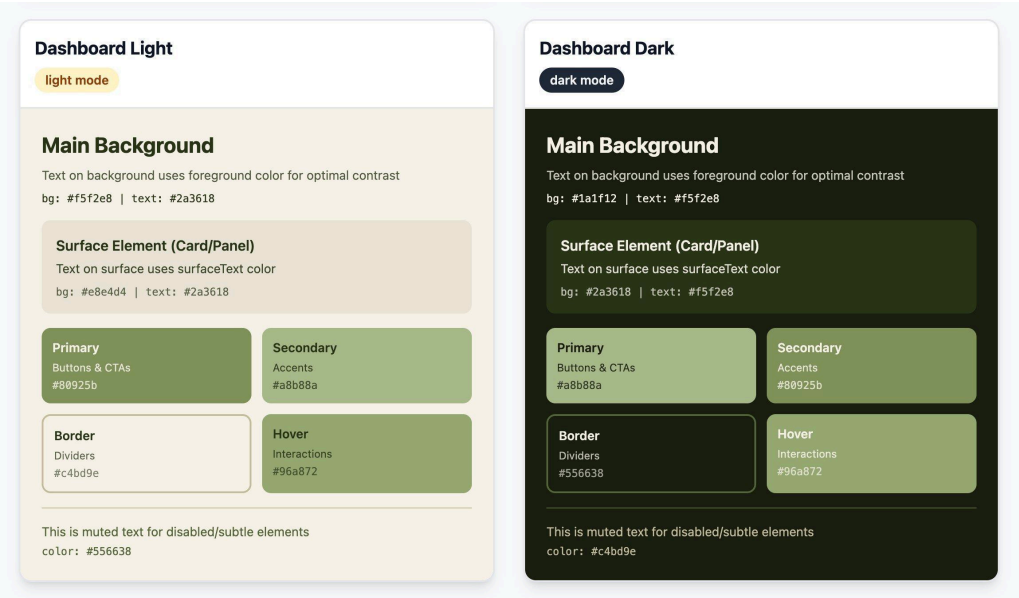


Figure 6.1 - Dashboard color palette (light and dark modes). The forest green (#1a1f12 dark / #f5f2e8 light) base with olive primary buttons establishes the platform's ambient nature-world identity. Implemented as CSS variables, not Tailwind utility classes.

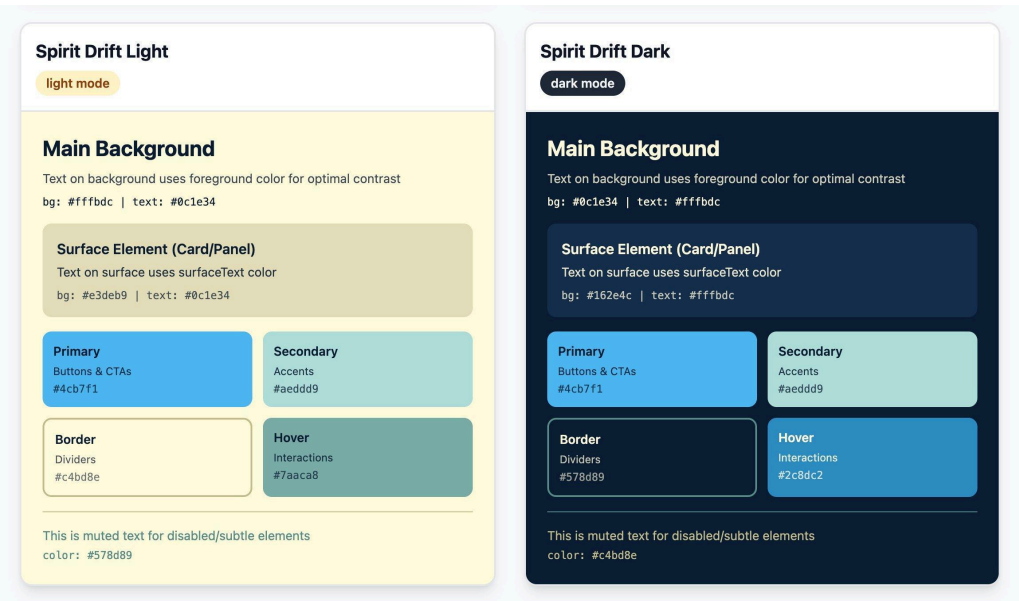


Figure 6.2 - Spirit Drift color palette (light and dark modes). Deep navy (#0c1e34) with sky blue primary (#4cb7f1) evokes the moonlit, celestial atmosphere of the game's wind-swept play environment.

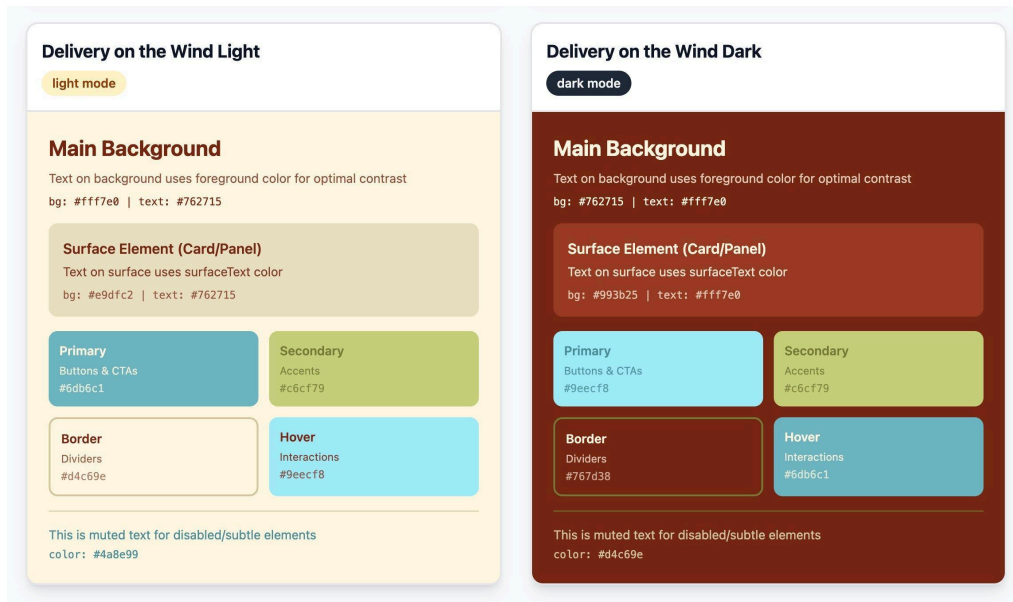


Figure 6.3 - Delivery on the Wind color palette (light and dark modes). Warm terracotta dark mode (#762715) with teal primary (#6db6c1) differentiates this game's earthy identity from the platform's nature-spirit aesthetic.

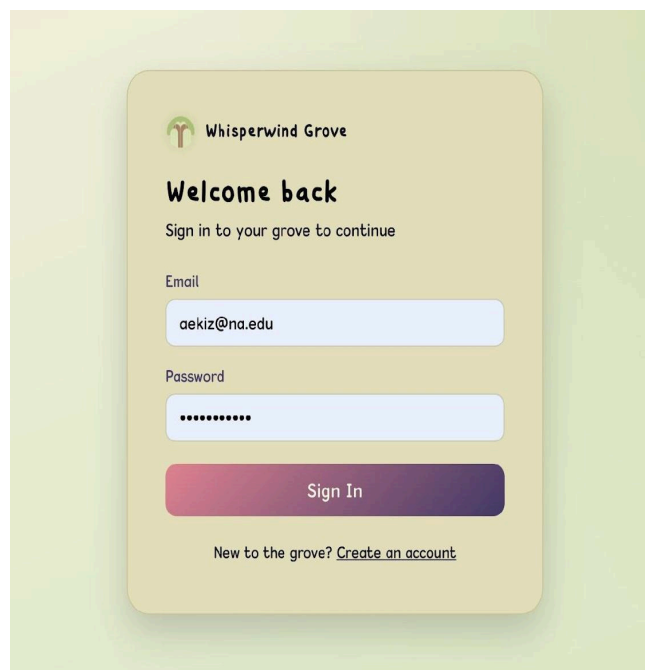


Figure 6.4 - The sign-in screen, using the platform's calming sage-green gradient background and the Whisperwind Grove logo. JWT tokens are stored under the `ww_auth` key in `localStorage` and retrieved by `authHeaders()` on every protected API call.

7. Phaser Integration

Phaser 3.90 is used for game worlds that require a canvas/WebGL rendering loop, sprite management, camera movement, pointer input, or scene-level state. React owns the outer shell and component lifecycle, while Phaser owns the frame-by-frame gameplay. The architectural boundary is firm and intentional: Phaser emits a completion state to React, and React performs all persistence.

7.1 Scene Configuration

Spirit Drift uses a Phaser scene with fixed logical dimensions of 1280×720 and responsive scaling through `Phaser.Scale.FIT`. This keeps the game proportional across screen sizes while React controls the surrounding game shell, rules screen, result overlay, and navigation buttons.

```
// frontend/src/game/createGame.ts
const config: Phaser.Types.Core.GameConfig = {
  type: Phaser.AUTO,
  parent,
  scale: {
    width: 1280,
    height: 720,
    mode: Phaser.Scale.FIT,
    autoCenter: Phaser.Scale.CENTER_BOTH,
  },
  backgroundColor: '#0b1020',
  scene: [scene],
}
const game = new Phaser.Game(config)
```

7.2 React/Phaser Boundary

The strongest architectural boundary is that Phaser scenes are responsible for gameplay only. When Spirit Drift reports a final score, the React wrapper receives the event and submits both a `ScoreSubmission` and a `GameSession` record to the backend. This prevents Phaser scenes from needing to know about JWT tokens, API routes, or backend schemas, keeping the game engine layer fully decoupled from the persistence layer.

7.3 Games Using Phaser

Spirit Drift, Delivery on the Wind, and Rise of the Half Moon use Phaser 3.90 for their rendering loop and gameplay mechanics. Spirit Sapling is implemented as a React component with CSS animation rather than as a Phaser scene, because its gameplay loop - energy timers, action buttons, growth stage transitions, and the chatbot panel - is better expressed in React's declarative model than in Phaser's imperative update cycle.

8. Backend Architecture

The backend is an Express 5 API written in TypeScript. The server loads environment variables, connects to MongoDB through Mongoose 9, creates the Express application, and mounts feature modules under `/api/v1`.

```
// backend/src/routes/index.ts
router.use('/auth',      authRoutes)
router.use('/progress',  progressRoutes)
router.use('/users',     usersRoutes)
router.use('/games',     scoresRoutes)
router.use('/games/half-moon', halfMoonRoutes)
router.use('/tts',       ttsRoutes)
router.use('/spirit-sapling', spiritSaplingRoutes)
router.use('/achievements', achievementsRoutes)
```

The backend is organized by domain. The auth module handles registration, login, and JWT issuing. The users module handles profile and avatar operations. The scores module handles score submission, leaderboard queries, personal bests, and recent scores. The sessions module handles per-run session history and the cross-game progress summary. The achievements module handles the badge catalog, earned badges, and reconciliation from existing history. The tts module proxies ElevenLabs Guardian speech. The spirit Sapling module handles Gemini kindness checks and Sapling chat responses. The halfMoon module provides the Gemini AI move proxy.

8.1 API Reference

Method	Route	Description
POST	/api/v1/auth/register	Create account
POST	/api/v1/auth/login	Authenticate and return JWT
GET	/api/v1/users/profile	Get an authenticated profile
PUT	/api/v1/users/profile	Update username
POST	/api/v1/games/:gameSlug/scores	Submit score
GET	/api/v1/games/:gameSlug/leaderboard	Public score leaderboard
GET	/api/v1/games/:gameSlug/me	Authenticated personal best
GET	/api/v1/games/:gameSlug/recent	Authenticated recent score records
POST	/api/v1/games/:gameSlug/sessions	Persist a session
GET	/api/v1/games/:gameSlug/sessions/me	Fetch session history
GET	/api/v1/games/progress/summary	Cross-game progress summary
GET	/api/v1/games/delivery-on-the-wind/leaderboard/fastest	Fastest delivery leaderboard
POST	/api/v1/games/half-moon/ai-move	Gemini move proxy
GET	/api/v1/achievements/catalog	Public achievement catalog
GET	/api/v1/achievements/me	Authenticated earned achievements
POST	/api/v1/tts/guardian	ElevenLabs Guardian Voice Synthesis
POST	/api/v1/spirit-sapling/kindness-check	Gemini kindness scoring
POST	/api/v1/spirit-sapling/chat	Sapling chat response

9. Database Design

Whisperwind Grove's persistence layer is implemented entirely in MongoDB through Mongoose 9. Old MySQL and Prisma/Postgres artifacts were removed from the codebase, so the active runtime persistence architecture is consistent and MongoDB-only.

9.1 Collections

Collection	Model	Purpose
users	User	Account identity, email, username, password hash, avatar
games	Game	Game catalog records keyed by slug
scoreSubmissions	ScoreSubmission	Individual score events powering recent score history
userHighScores	UserHighScore	One best score per user per game; powers leaderboard ranking
gameSessions	GameSession	Full run history with game-specific metadata
badges	Badge	Achievement definitions and criteria
userBadges	UserBadge	Earned achievements per user; written with upsert idempotency

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
badges	-	36.86 kB	2.17 kB	8	270.00 B	2	73.73 kB
games	-	36.86 kB	776.00 B	6	129.00 B	2	73.73 kB
gamesessions	-	36.86 kB	9.05 kB	18	502.00 B	4	147.46 kB
scoresubmissions	-	36.86 kB	3.69 kB	15	245.00 B	3	110.59 kB
userbadges	-	36.86 kB	546.00 B	6	91.00 B	2	73.73 kB
usergameprogresses	-	4.10 kB	0 B	0	0 B	2	8.19 kB
userhighscores	-	36.86 kB	306.00 B	3	102.00 B	3	110.59 kB
users	-	36.86 kB	799.00 B	4	199.00 B	3	110.59 kB
voiceclips	-	4.10 kB	0 B	0	0 B	2	8.19 kB

Figure 9.1 - MongoDB Atlas Data Explorer showing the live whisperwind_grove database under the Reverie cluster. All nine collections are visible: badges (8 documents), games (6), game sessions (18), score submissions (15), userbadges (6), usergameprogresses (0), userhighscores (3), users (4), and voiceclips (0). The index counts confirm that the compound indexes defined in the Mongoose schemas are active in the hosted database. The user game progresses, and voice clip collections are scaffolded for future features.

9.2 Flexible Session Schema

The GameSession model intentionally supports common fields and optional game-specific fields through nullable columns. This avoids forcing every game into the same rigid schema: a timed delivery run stores completionTimeSeconds and deliveriesCompleted, while a Half Moon run stores levelReached, totalCardPoints, moonScore, and won. A Spirit Sapling session stores guardianId and fruitsCollected.

```
// backend/src/models/GameSession.ts
const GameSessionSchema = new Schema<IGameSession>({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  gameSlug: { type: String, required: true },
  score: { type: Number, default: null },
  completed: { type: Boolean, required: true, default: false },
  won: { type: Boolean, default: null },
  levelReached: { type: Number, default: null },
  completionTimeSeconds: { type: Number, default: null },
  deliveriesCompleted: { type: Number, default: null },
  guardianId: { type: String, default: null },
  totalCardPoints: { type: Number, default: null },
});
```

```
moonScore:          { type: Number, default: null },
fruitsCollected:    { type: Number, default: null },
date:                { type: Date, default: Date.now },
}, { timestamps: { createdAt: true, updatedAt: false } })
```

9.3 Indexing Strategy

The GameSession model defines three compound indexes to support the most common query patterns without requiring full collection scans.

```
// Fetching a player's recent sessions for one game
GameSessionSchema.index({ userId: 1, gameSlug: 1 })
// Building cross-game progress summaries
GameSessionSchema.index({ gameSlug: 1, createdAt: -1 })
// Ranking fastest Delivery on the Wind completions
GameSessionSchema.index({ gameSlug: 1, completed: 1, completionTimeSeconds: 1 })
```

9.4 Score and Session Synchronization

The project uses two complementary persistence concepts. ScoreSubmission records score events and powers the recent-score history. GameSession records richer run metadata and powers progress and history views. For score-based games, session creation also updates score records and high scores through a single service call, ensuring games such as Spirit Sapling are visible in both progress history and score history.

```
// backend/src/modules/sessions/sessions.service.ts
if (score !== null && score !== undefined && game) {
  await ScoreSubmission.create({
    userId: userOid, gameId: game._id, score,
    durationMs: rest.completionTimeSeconds
      ? rest.completionTimeSeconds * 1000 : null,
    metadata: { source: 'session', gameSlug,
      completed: rest.completed, won: rest.won ?? null,
      guardianId: rest.guardianId ?? null },
  })

  await UserHighScore.findOneAndUpdate(
    { userId: userOid, gameId: game._id },
    { $max: { score }, $setOnInsert: { achievedAt: new Date() } },
    { upsert: true }
  )
}
```

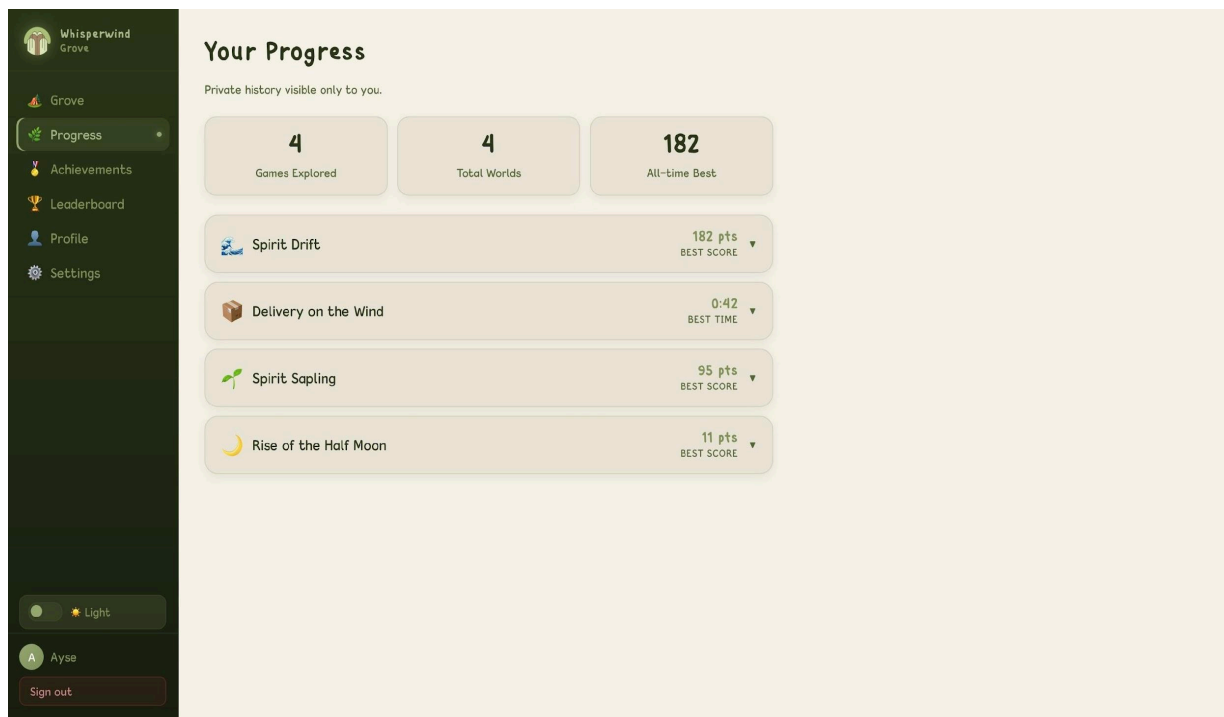


Figure 9.2 - The Progress page summary view, displaying UserHighScore records for all four games: Spirit Drift 182 pts, Delivery on the Wind 0:42 best time, Spirit Sapling 95 pts, Rise of the Half Moon 11 pts. The three header stats (4 Games Explored, 4 Total Worlds, 182 All-time Best) are derived from the /api/v1/games/progress/summary aggregation endpoint.

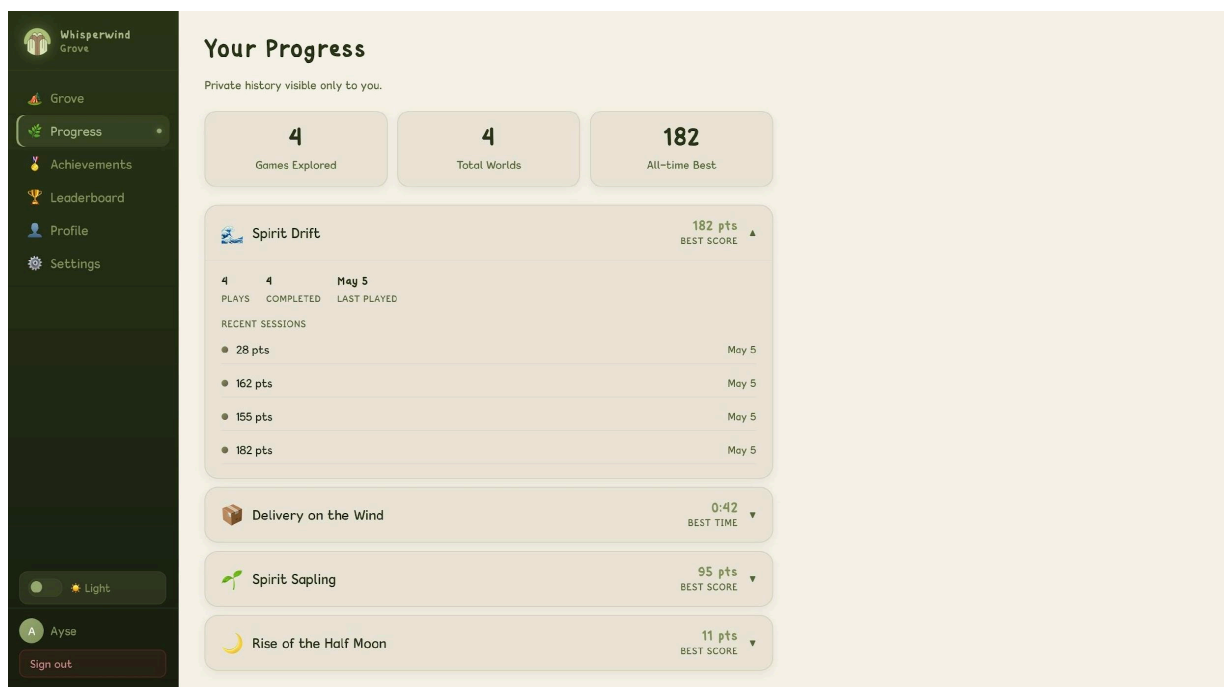


Figure 9.3 - The Progress page with the Spirit Drift row expanded, showing the ScoreSubmission history: 4 plays, 4 completed, last played May 5. Recent sessions (28 pts, 162 pts, 155 pts, 182 pts) demonstrate that every individual run is persisted as a distinct ScoreSubmission document - not just the personal best - enabling full session history display.

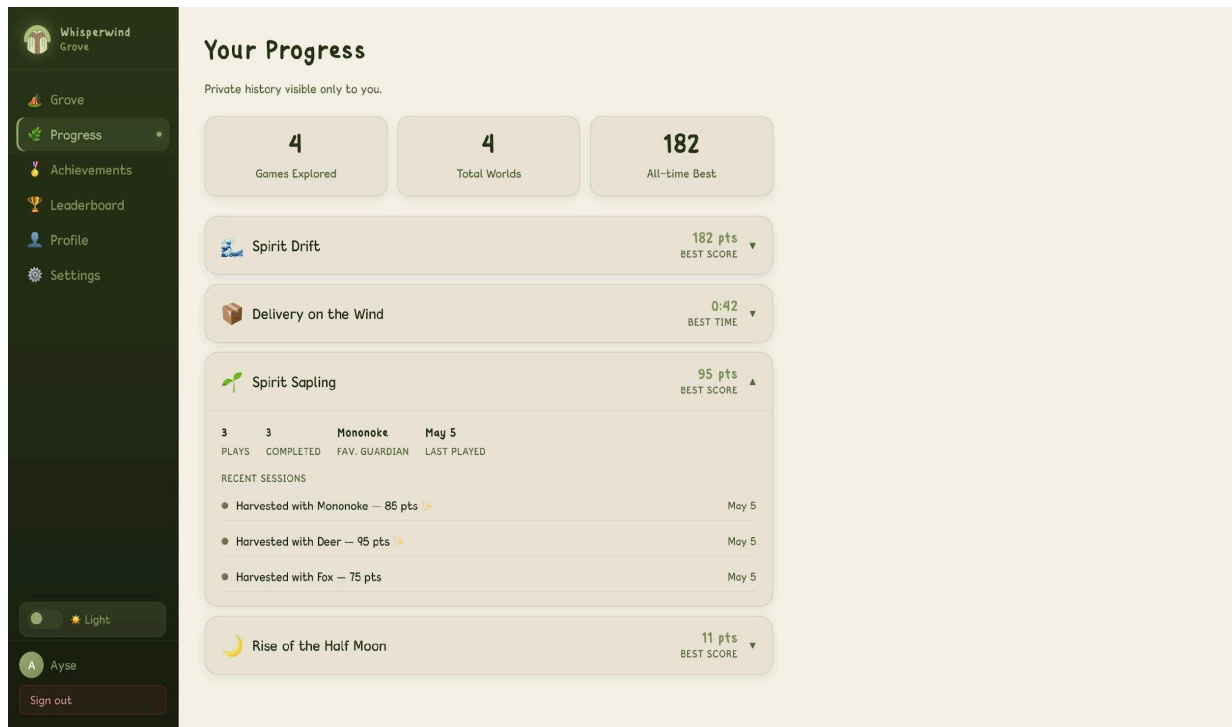


Figure 9.4 - The Spirit Sapling row expanded with 3 recent sessions: Harvested with Mononoke - 85 pts, Harvested with Deer - 95 pts, Harvested with Fox - 75 pts. The Fav. Guardian field (Mononoke) is derived from the most frequent guardianId in the player's Spirit Sapling GameSession records. This confirms that guardianId is correctly persisted and queryable from the GameSession collection.

10. Achievement System

10.1 Badge Definitions

The current achievement catalog contains eight badges spanning all four games. Spirit Sapling contributes Peach Tree Unlocked, Persimmon Tree Unlocked, Pear Tree Unlocked, Apple Tree Unlocked, and Grove Orchard Keeper (all four trees). Delivery on the Wind contributes to One-Minute Courier (completionTimeSeconds under 60). Spirit Drift contributes Windcatcher Adept (score over 200). Rise of the Half Moon contributes Moonlit Strategist (score of at least 50).

10.2 Idempotent Unlocking

The backend uses MongoDB upsert semantics to prevent duplicate achievement rows. The `$setOnInsert` operator ensures that if a matching `UserBadge` document already exists, no fields are overwritten, and the upsert is a no-op. The service returns the badge payload only when `upsertedCount` is 1, meaning the badge is genuinely new. This payload drives the `AchievementToast` component on the frontend.

```
// backend/src/modules/achievements/achievements.service.ts
const result = await UserBadge.updateOne(
  { userId: new Types.ObjectId(userId), badgeId: badge._id },
  { $setOnInsert: { awardedAt: new Date() } },
  { upsert: true }
)

if (result.upsertedCount === 0) return null // already earned
```

10.3 Catalog Fallback

The Achievements page uses a protected route for earned achievements but falls back to the public catalog endpoint if the backend is temporarily unavailable or the token is stale. This prevents the page from rendering blank under error conditions.

```
// frontend/src/lib/api.ts
export async function getAchievements() {
  try {
    const res = await fetch(apiUrl('/achievements/me'), {
      headers: authHeaders(),
    })
    if (!res.ok) return getAchievementCatalog()
    return res.json()
  } catch {
    return getAchievementCatalog()
  }
}
```

10.4 Reconciliation from Existing History

The achievement listing service also checks existing sessions and high scores when building the earned badges response. If the game history proves that a badge should have been unlocked - for example, if a GameSession with guardianId 'deer' and completed: true exists - the backend restores the missing UserBadge row. This makes the achievement state recoverable from the MongoDB history even if an earlier write failed.

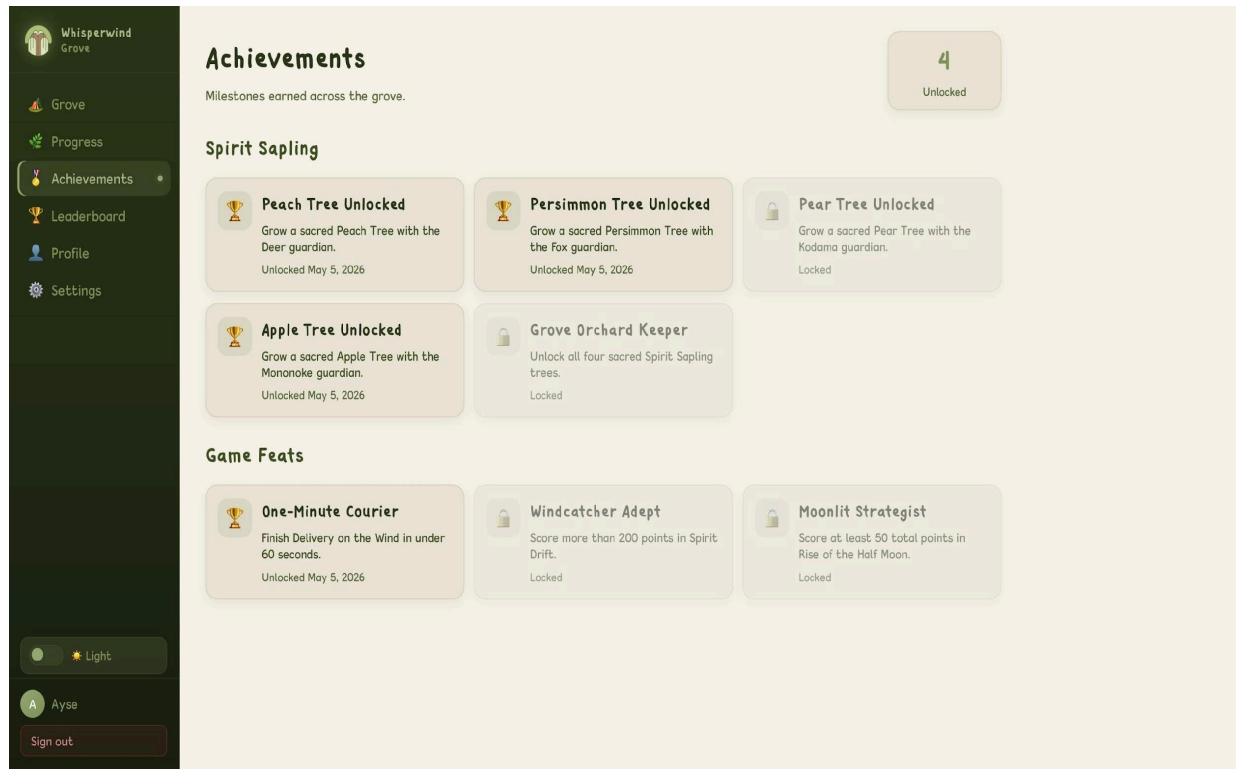


Figure 10.1 - The Achievements page after additional Spirit Sapling sessions, now showing 4 Unlocked badges. Peach Tree Unlocked, Persimmon Tree Unlocked, and Apple Tree Unlocked are all earned (gold trophies with May 5, 2026 dates), while Pear Tree, Grove Orchard Keeper, Windcatcher Adept, and Moonlit Strategist remain locked. The counter in the top-right reflects live UserBadge document counts from the MongoDB query.

10.5 Leaderboard

The Grove Leaderboard aggregates the top UserHighScore records per game and the fastest GameSession completion times for Delivery on the Wind, presenting them as a unified public ranking page. Each game section shows a ranked player list with scores, times, and dates. The leaderboard is powered by the /api/v1/games/:gameSlug/leaderboard endpoint for score-based games and the /api/v1/games/delivery-on-the-wind/leaderboard/fastest endpoint for timed completions.

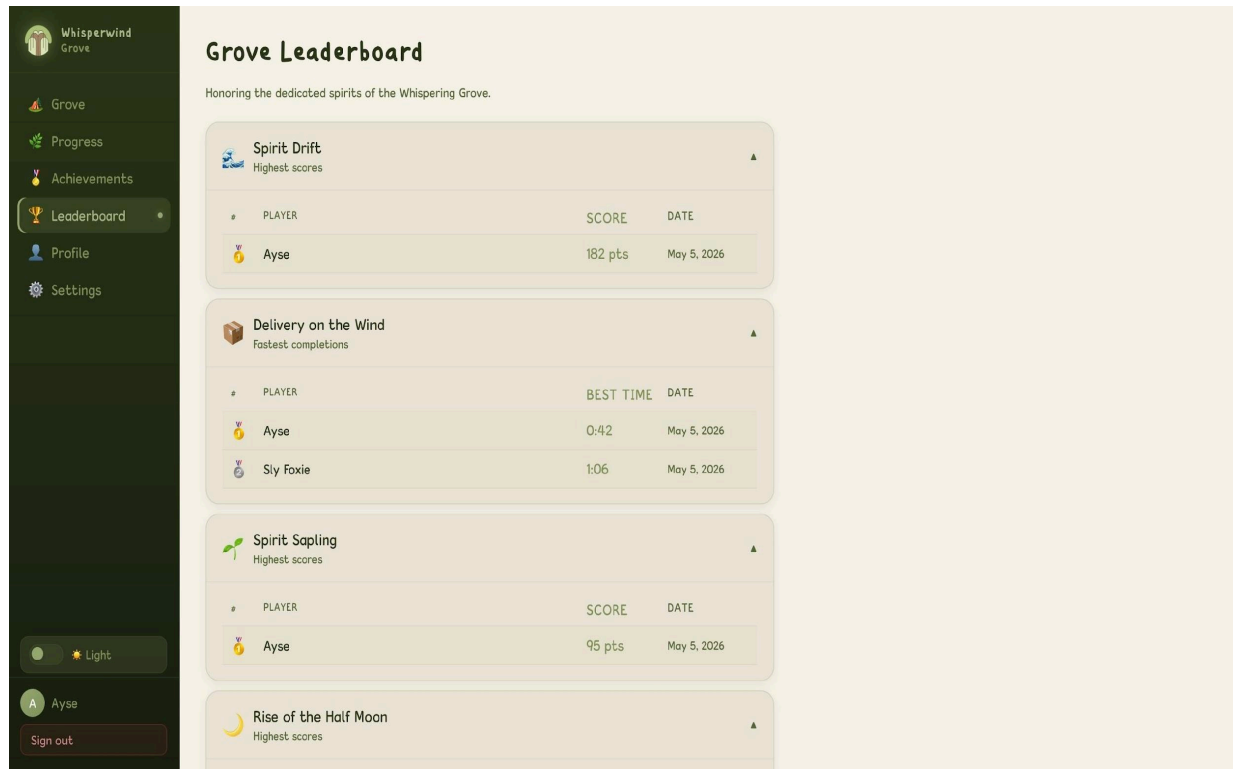


Figure 10.2 - The Grove Leaderboard page. Spirit Drift shows Ayse at #1 with 182 pts. Delivery on the Wind shows Ayse first at 0:42 and Sly Foxie second at 1:06, demonstrating that the fastest-time leaderboard correctly uses completionTimeSeconds ascending from the GameSession collection. Spirit Sapling shows Ayse at 95 pts. Rise of the Half Moon is shown in a collapsed state below. The leaderboard subtitle - 'Honoring the dedicated spirits of the Whispering Grove' - reinforces the platform's narrative identity.

11. Game Design and Mechanics

Each of the four games in Whisperwind Grove is designed to contribute a distinct emotional and mechanical dimension to the platform's shared progression system while remaining creatively independent in its visual identity and gameplay loop.

11.1 Spirit Drift

Spirit Drift is the platform's arcade score game, built entirely in Phaser 3.90. It uses a 60-second timer, multiple spirit types with different point values, combo multipliers with decay, cursed objects that penalize misclicks, and three-phase difficulty escalation that increases spawn rates and movement speed as time progresses. Wind Surge special events briefly double all points. The combo state is stored inside the Phaser scene, and the React wrapper submits score and session results after completion.

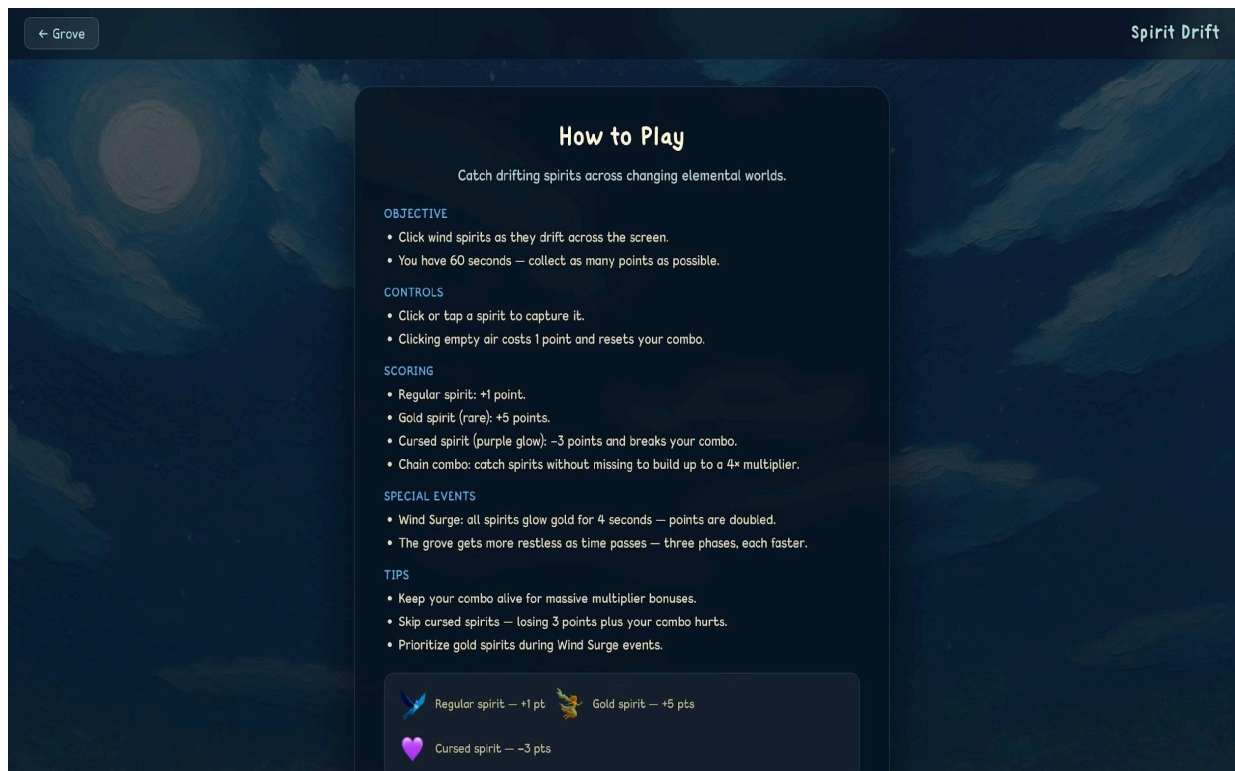


Figure 11.1 - The Spirit Drift rules screen, displayed over the game's atmospheric oil-painting sky background. The scoring system (Regular spirit +1, Gold spirit +5, Cursed spirit -3 plus combo reset), three-phase escalation, and Wind Surge event mechanics are documented before gameplay begins.

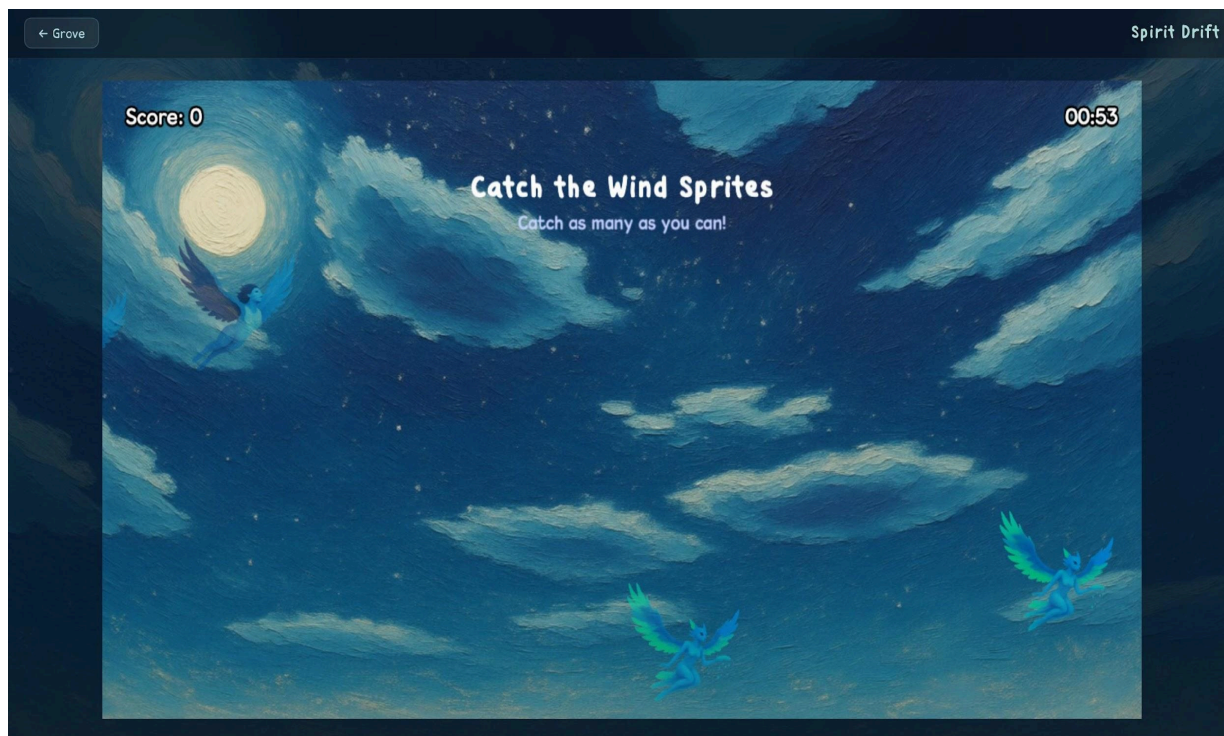


Figure 11.2 - Spirit Drift active gameplay. The Phaser-rendered canvas shows three wind spirits drifting across the moonlit sky, with score counter (top-left) and countdown timer (top-right). The minimal HUD preserves the visual clarity of the play space.

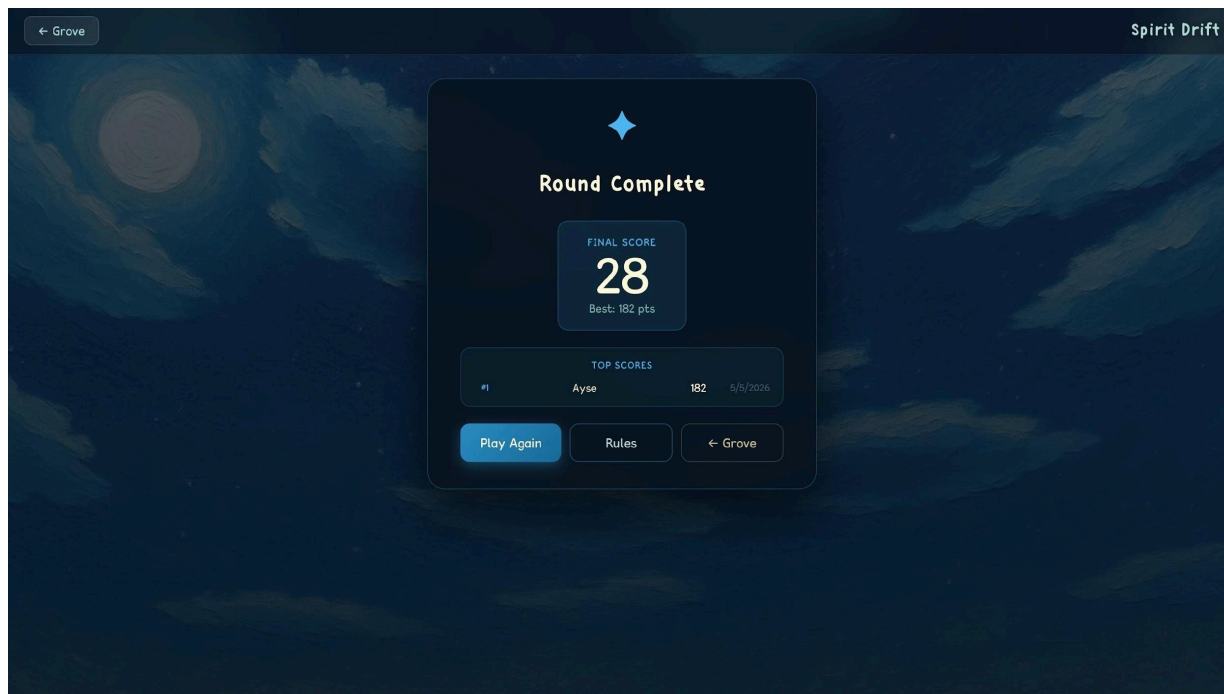


Figure 11.3 - Spirit Drift result screen. The Round Complete overlay shows the session score (28), personal best (182 pts), and the Top Scores leaderboard. Leaderboard data is fetched fresh after persistence, ensuring the result screen always reflects the latest state.

11.2 Spirit Sapling

Spirit Sapling is a nurturing game with six growth stages. The player chooses one of four guardians - Deer, Fox, Kodama, or Mononoke - and uses water, sun, and voice interactions to grow the sapling from seedling to fruit-bearing sacred tree. Unlike the other three games, Spirit Sapling is implemented as a React component rather than a Phaser scene, because its interaction loop - energy timers, care action buttons, growth stage transitions, and the Gemini chatbot panel - is better expressed in React's declarative model.



Figure 11.4 - The Spirit Sapling welcome screen, showing the three-stage gameplay flow: Choose your guardian → Nurture the sapling → Collect the harvest. Scoring metrics (Saplings grown, Fruits collected, Harmony bonus, Final score) communicate the depth of the progression system before gameplay begins.

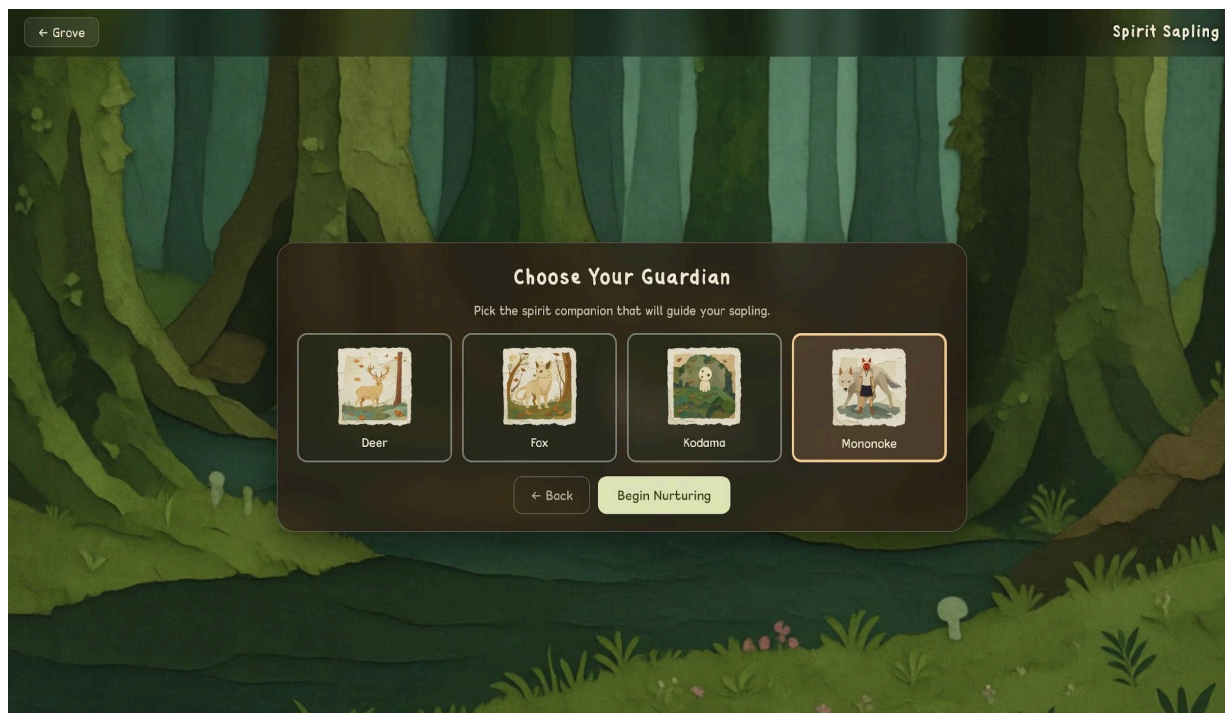


Figure 11.5 - The Choose Your Guardian screen. Each guardian carries a distinct visual identity, ElevenLabs voice profile, and sacred tree type (Deer → Peach, Fox → Persimmon, Kodama → Pear, Mononoke → Apple). Guardian selection is stored in the component state and persisted in the `GameSession`'s `guardianId` field.

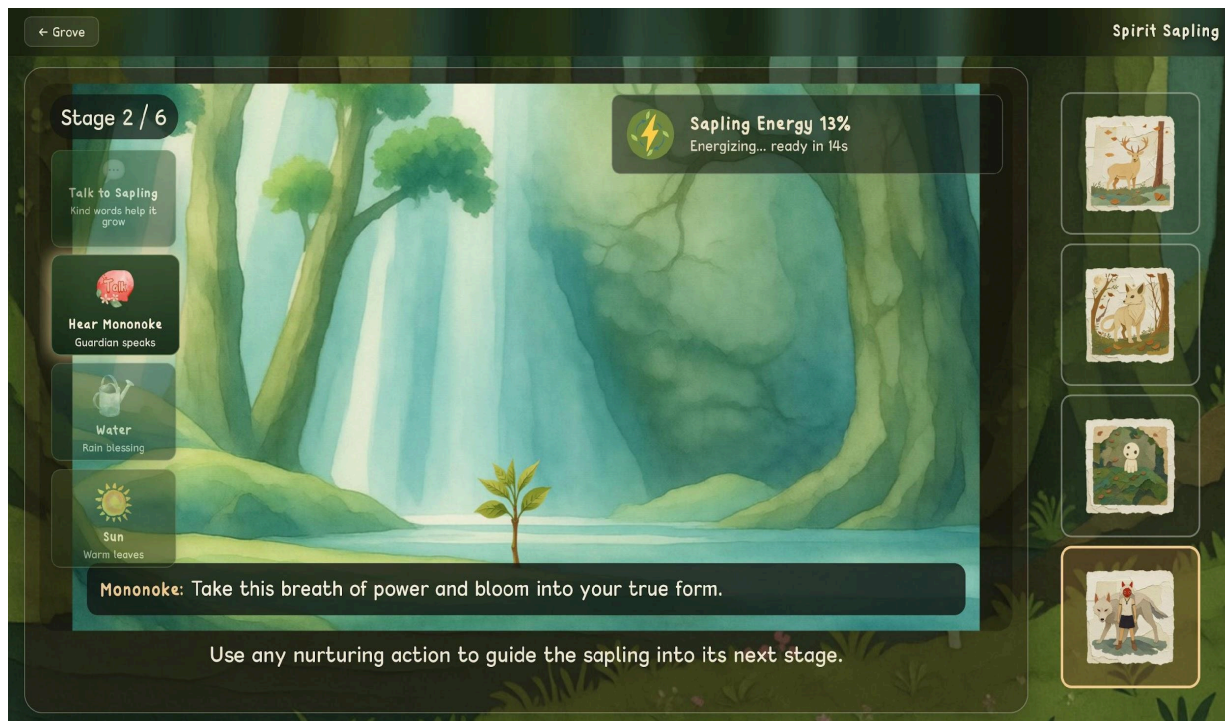


Figure 11.6 - Mononoke's voice line displayed in-canvas: 'Mononoke: Take this breath of power and bloom into your true form.' This text is displayed when the Hear Guardian button triggers the ElevenLabs TTS API through the backend, which streams audio to the browser while showing the narrative message.



Figure 11.7 - Spirit Sapling at Stage 5/6, showing the sapling's progression to a fully-canopied tree - a visual contrast to the seedling in Figure 11.6 that makes the six-stage growth arc tangible.

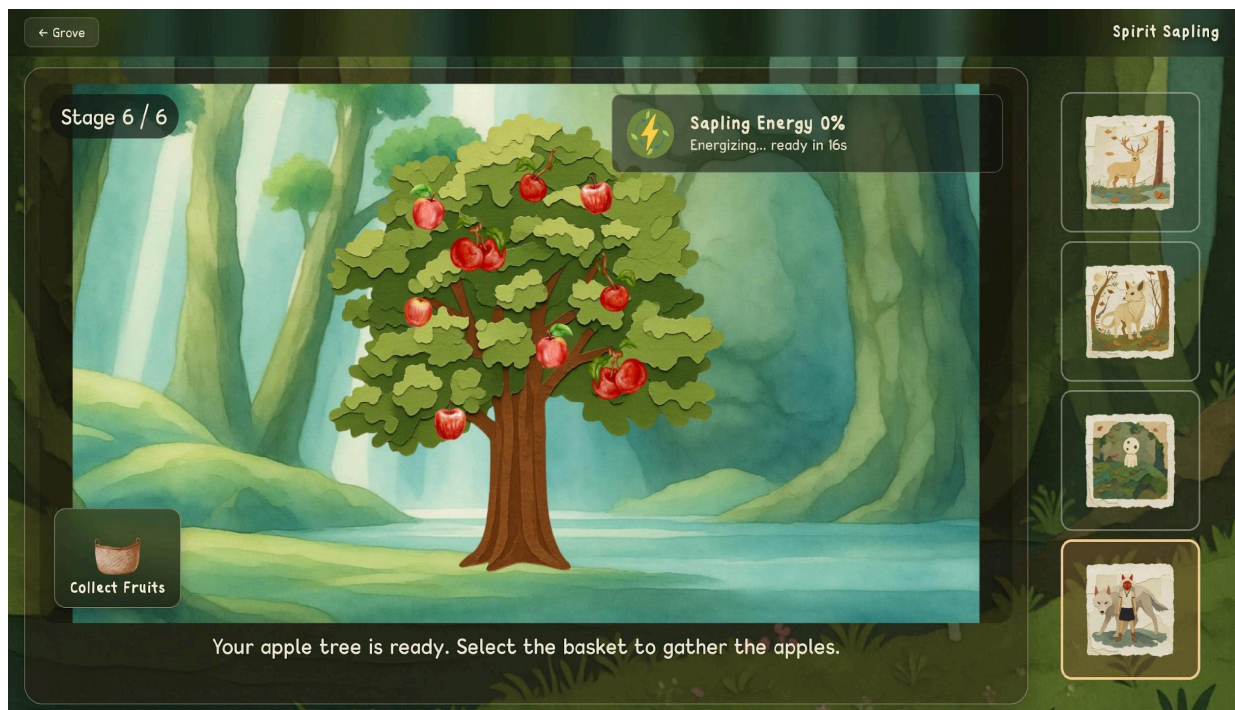


Figure 11.8 - Spirit Sapling at Stage 6/6, the fruit-bearing completion stage. The Collect Fruits basket button is positioned as a bottom-left overlay within the game stage - a layout change made after the original implementation caused the button to render outside the viewport at certain screen sizes. The harvest result screen (scoring: Fully grown +50, Harvest +10, Harmony +20, Kind words $\times 1 + 5 = 85$ pts) follows, accompanied by the Apple Tree Unlocked Achievement Toast confirming the UserBadge write to MongoDB succeeded.

11.3 Delivery on the Wind

Delivery on the Wind is a timed top-down navigation game built in Phaser 3.90. The player character Kiki must deliver four packages to their correct destinations across an illustrated village map before the clock runs out. The game uses grid-based movement and camera targeting, with Package and House entities modeled as Phaser objects. Completion time and deliveries completed are persisted in GameSession. The fastest-time leaderboard is powered by a MongoDB aggregation over completed sessions filtered by completionTimeSeconds.

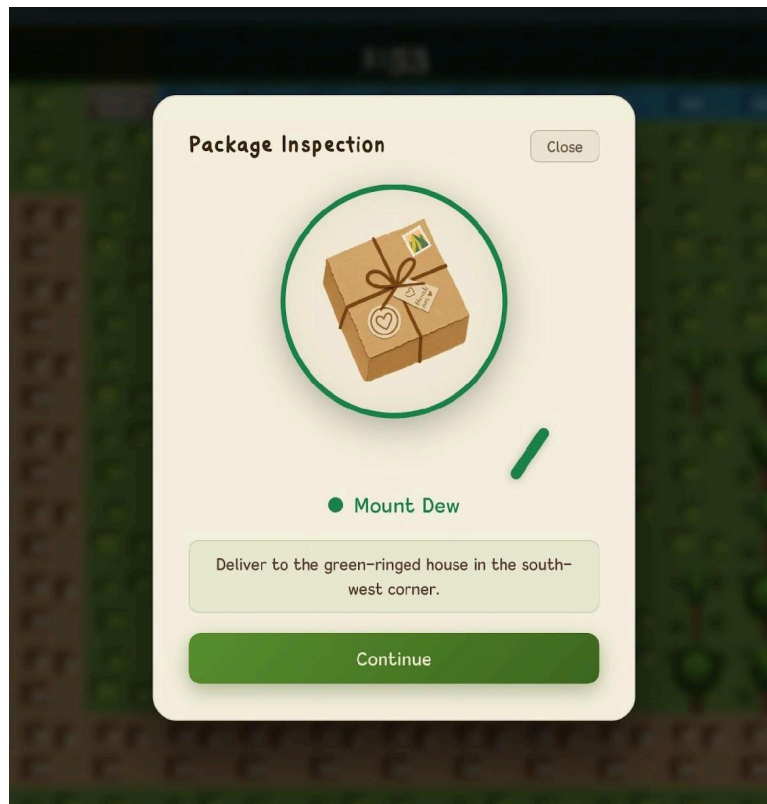


Figure 11.11 - The Package Inspection modal, displaying the parcel type, color-coded destination ring (green = Mount Dew, south-west corner), and a Continue button. This modal is triggered on package pickup and provides narrative context before the player navigates to the delivery address.



Figure 11.12 - The in-game 'All Delivered!' overlay: 4/4 packages delivered with 0:43 time remaining. The tile-map village and Kiki sprite are visible in the Phaser canvas behind the celebration overlay.

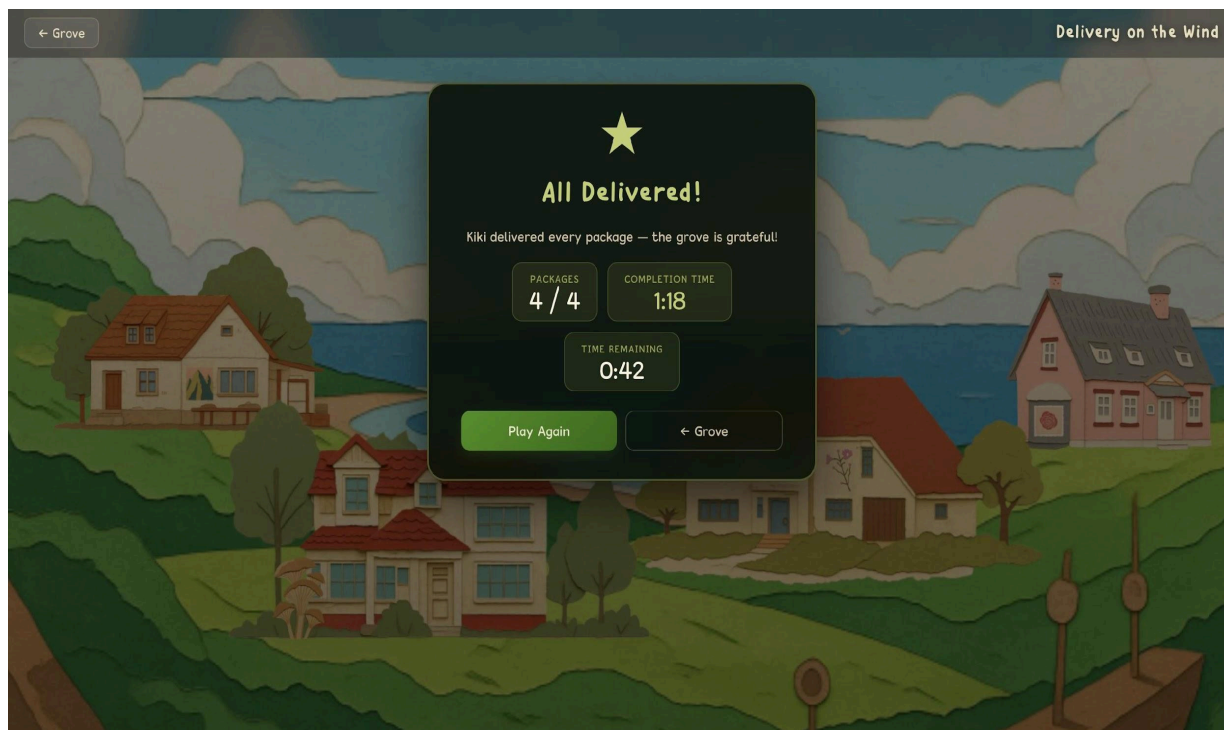


Figure 11.13 - Delivery on the Wind result screen. Completion Time 1:18 and Time Remaining 0:42 are shown - a run that qualifies for the One-Minute Courier achievement (completionTimeSeconds < 60). These values are persisted in the GameSession's completionTimeSeconds field.

11.4 Rise of the Half Moon

Rise of the Half Moon is a three-level moon-phase card placement strategy game. The player must win all three levels consecutively; losing any level ends the run and persists a failed GameSession with completed: false and won: false. The deck contains 8 lunar phases (New Moon through Waning Crescent, four copies each), and players draw 3 cards to start. Scoring mechanics include same-phase adjacency bonuses, complementary-phase bonuses, Moon Cycle chains, and chain stealing.

The game offers two AI modes: Local AI, which uses a deterministic strategy engine, and Gemini AI, which uses the backend Gemini proxy to suggest moves. In Gemini AI mode, every move returned by the model is validated by the frontend before being accepted into the game state.

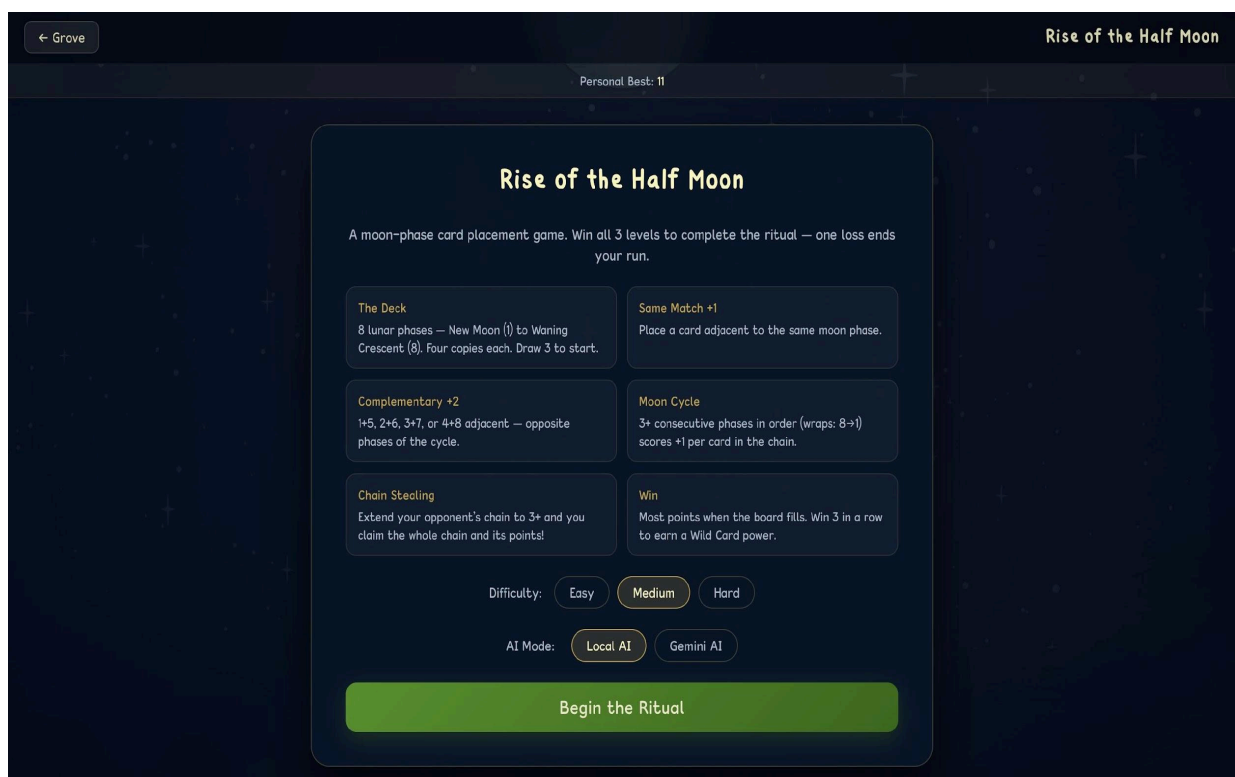


Figure 11.14 - The Rise of the Half Moon rules screen. Difficulty (Easy/Medium/Hard) and AI Mode (Local AI/Gemini AI) are selectable before play. Personal Best: 11 is shown in the header bar. The full rule set is presented on cards matching the game's deep navy aurora palette.

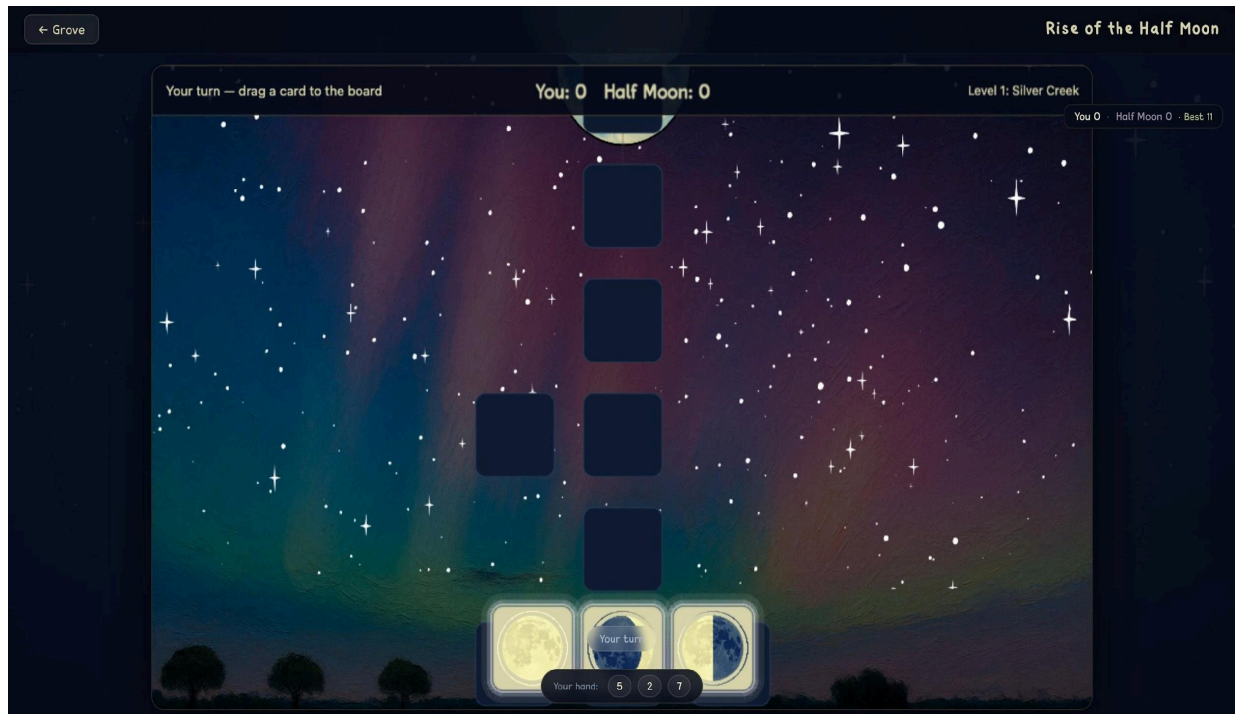


Figure 11.15 - Rise of the Half Moon at the start of Level 1 (Silver Creek). The Phaser-rendered board shows five empty card slots arranged vertically against the aurora-lit night sky. The player's hand (cards 5, 2, and 7 - Gibbous Waxing, New Moon, and Waning Crescent phases) is displayed at the bottom. The drag-to-place interaction is managed by Phaser's pointer input system.

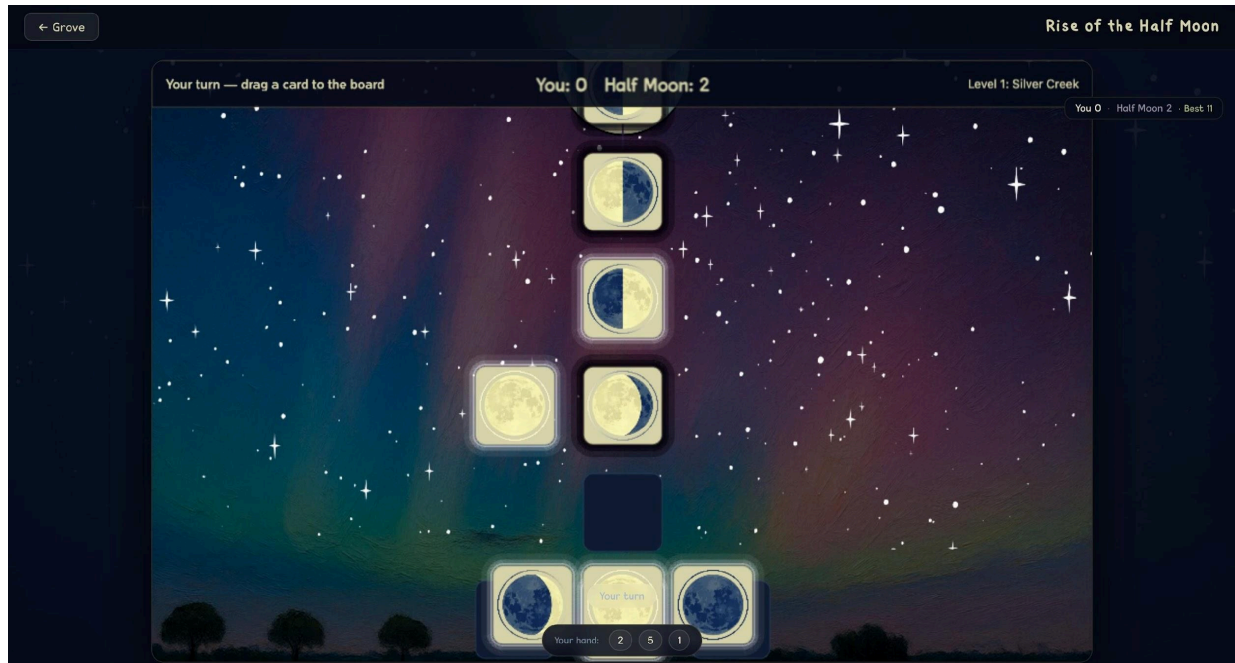


Figure 11.16 - Rise of the Half Moon mid-game at Level 1. Four moon-phase cards have been placed on the board, with the AI (Half Moon) having scored 2 points and the player on 0. The illustrated moon-phase cards - Full Moon, Half Moon, Waning Gibbous, and Full Moon - show the visual scoring mechanic: adjacent same-phase and complementary-phase placements are evaluated at board-fill. One empty slot and the player's three remaining hand cards are visible at the bottom.

12. AI Integration

Whisperwind Grove uses two external AI services as gameplay mechanics: Google Gemini 2.0 Flash for Spirit Sapling kindness evaluation and Rise of the Half Moon move suggestions, and ElevenLabs for guardian voice synthesis. All AI API keys are stored in backend environment variables and never exposed to the frontend.

12.1 Gemini Kindness Evaluation - Spirit Sapling

When a player submits a message in the Talk to Sapling panel, the frontend sends the text to the `/api/v1/spirit-sapling/kindness-check` endpoint. The backend validates message length, constructs a constrained prompt instructing Gemini to respond only with valid JSON, calls Gemini 2.0 Flash, strips any markdown fences if necessary, parses the response, and returns a structured payload.

```
// Gemini response structure
{
  approved: boolean
  reason: string
  growthBoost: 0 | 1 | 2
}
```

The growthBoost value is a bounded integer, not a free-form text result. A growthBoost of 1 reduces the energy recharge timer by 4 seconds; a growthBoost of 2 reduces it by 8 seconds. The frontend applies this through the following state update:

```
const BASE_ENERGY_RECHARGE_SECONDS = 20
const MIN_ENERGY_RECHARGE_SECONDS = 8
const KINDNESS_RECHARGE_REDUCTION_SECONDS = 4

const handleKindnessApproved = (growthBoost: number) => {
  setTalkBoostTotal((prev) => prev + growthBoost)
  setEnergyRechargeSeconds((current) =>
    Math.max(
      MIN_ENERGY_RECHARGE_SECONDS,
      current - growthBoost * KINDNESS_RECHARGE_REDUCTION_SECONDS,
    )
  )
}
```

This produces the intended progression: baseline 20 seconds → one approved message → 16 seconds → another approved message → 12 seconds → minimum cap of 8 seconds. A growth boost of 2 from a particularly expressive message can jump the timer down by 8 seconds in a single exchange.

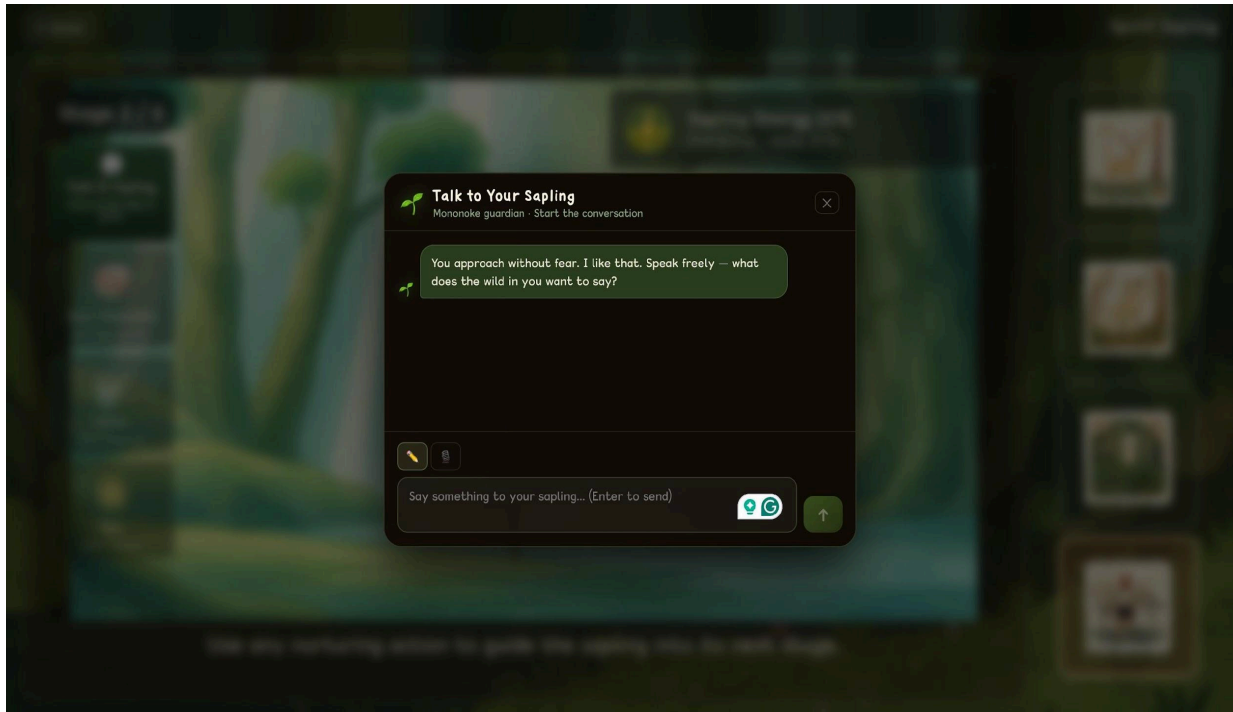


Figure 12.1 - The Talk to Sapling panel (initial state). The Mononoke Guardian's opening message is shown as a chat bubble. The Gemini logo is visible in the text input area, indicating the AI-powered evaluation layer. The game canvas is visible and blurred behind the panel.

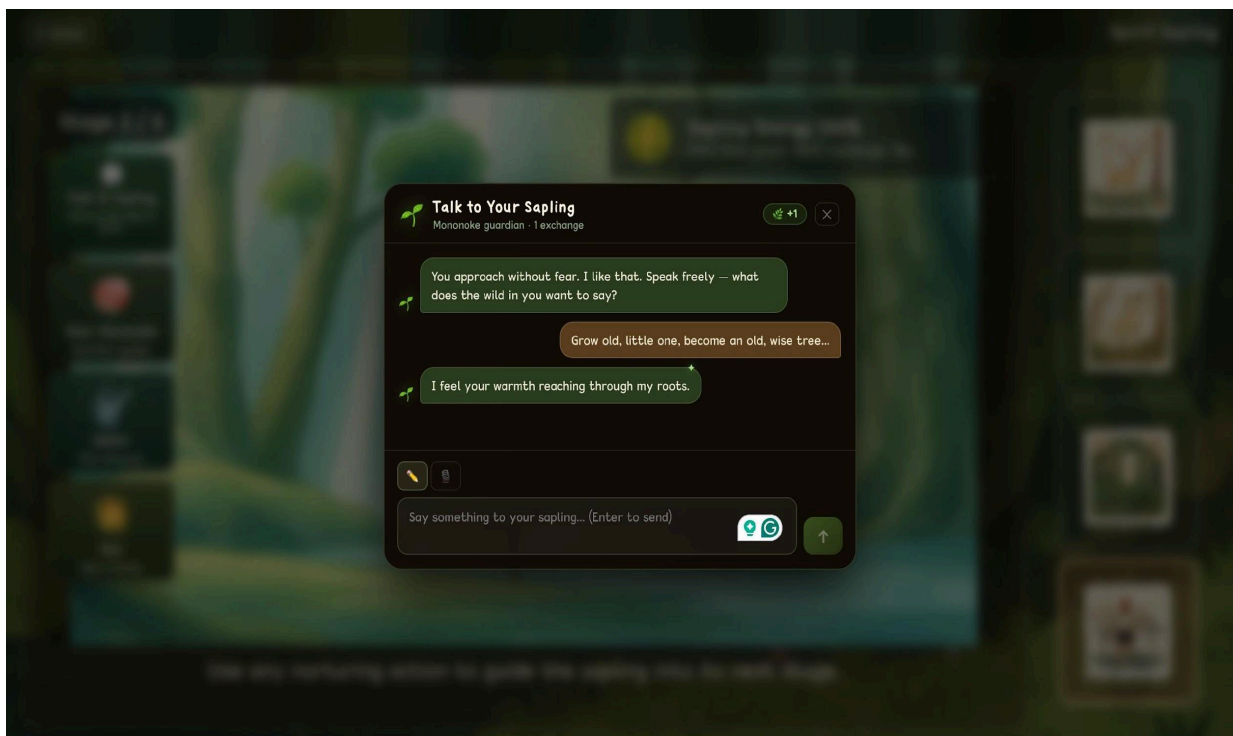


Figure 12.2 - The Talk to Sapling panel after an approved kind message ('Grow old, little one, become an old, wise tree...'). The sapling responds: 'I feel your warmth reaching through my roots.' The '🌱 +1' badge in the panel header confirms growthBoost: 1 was returned by Gemini and applied to the recharge timer.

12.2 Gemini Move Proxy - Rise of the Half Moon

In Gemini AI mode, the Half Moon frontend sends the AI's current hand, the placed cards on the board, the current level, and all empty space IDs to the `/api/v1/games/half-moon/ai-move` endpoint. The backend sanitizes all numeric values, constructs a concise strategy prompt, calls Gemini, parses JSON defensively, and returns either a move object or null. The frontend then applies a second layer of validation before mutating the game state.

```
// frontend - move validation before acceptance
function validateMove(
  raw: { spaceId: number; phase: number },
  hand: Phase[],
  emptySpaceIds: number[],
): GeminiMove | null {
  if (!emptySpaceIds.includes(raw.spaceId)) return null
  if (!Number.isInteger(raw.phase) || raw.phase < 1 || raw.phase > 8) return null
  if (!hand.includes(raw.phase as Phase)) return null
  return { spaceId: raw.spaceId, phase: raw.phase as Phase }
}
```

This double-validation pattern is a critical AI safety boundary: Gemini is allowed to propose, but deterministic game logic decides whether the proposal is legal. The game is never placed in an invalid state by a malformed or hallucinated model response.

12.3 ElevenLabs Guardian Voice

Guardian voice synthesis is handled server-side. The backend stores voice IDs for all four guardians as environment variables, constructs the ElevenLabs request with the `eleven_multilingual_v2` model, and streams the audio response to the frontend. Per-guardian voice settings, such as stability, similarity, style, and speaker boost are configurable in the backend without any frontend changes.

```
// backend - guardian voice ID map
const VOICE_IDS: Record<GuardianId, string> = {
  deer:    process.env.ELEVENLABS_VOICE_ID_DEER    || '',
  fox:     process.env.ELEVENLABS_VOICE_ID_FOX     || '',
  kodama:  process.env.ELEVENLABS_VOICE_ID_KODAMA  || '',
  mononoke: process.env.ELEVENLABS_VOICE_ID_MONONOKE || ''
}

const modelId = process.env.ELEVENLABS_MODEL_ID || 'eleven_multilingual_v2'
```

The API key and all voice IDs remain in `backend/.env` and are never present in frontend code or browser network requests.

13. Security and Configuration

The project uses environment variables for all sensitive configuration. Passwords are hashed with bcrypt before storage. JWTs are verified in authMiddleware on every protected route. All Gemini and ElevenLabs API keys are stored exclusively on the backend. Helmet sets security-related HTTP response headers. CORS is configured for frontend/backend integration.

The following environment variables must be defined in backend/.env:

```
MONGODB_URI,  
JWT_SECRET,  
GEMINI_API_KEY,  
ELEVENLABS_API_KEY,  
ELEVENLABS_MODEL_ID,  
ELEVENLABS_VOICE_ID_DEER,  
ELEVENLABS_VOICE_ID_FOX,  
ELEVENLABS_VOICE_ID_KODAMA, and  
ELEVENLABS_VOICE_ID_MONONOKE.
```

The frontend requires only VITE_API_BASE to point to the backend origin in production deployments.

14. Testing and Verification

The primary verification commands for the project are:

```
npm --prefix frontend run build    # Production bundle validation
npm --prefix frontend run lint     # ESLint static analysis
cd backend && npx tsc --noEmit      # TypeScript type checking
```

The recommended manual test matrix covers all major platform paths, including cross-game persistence, AI integration, TTS behavior, and error fallback conditions.

Area	Test	Expected Result
Auth	Register and log in	JWT saved as ww_auth; dashboard opens
API base	Change VITE_API_BASE	Auth and game calls use the new base path
MongoDB	Complete any game	GameSession document created
Scores	Complete score-based game	ScoreSubmission and UserHighScore updated
Spirit Sapling	Kind message submitted	Recharge time decreases by 4s or 8s per growthBoost value
Spirit Sapling	Finish with a guardian	Matching sacred tree achievement unlocks
Delivery	Finish under 60 seconds	One-Minute Courier achievement unlocks
Spirit Drift	Score over 200	Windcatcher Adept achievement unlocks
Half Moon	Score at least 50	Moonlit Strategist achievement unlocks
Half Moon	Lose any level	Failed session persisted with completed: false and won: false
Achievements	Backend unavailable or token expired	Catalog still displays; no blank screen
TTS	Click Hear Guardian for each spirit	Backend logs guardian name and voice suffix; audio plays
Half Moon AI	Gemini returns an invalid move	validateMove returns null; local AI fallback used

15. Engineering Challenges and Solutions

15.1 Keeping MongoDB as the Only Persistence Path

Earlier project versions contained old MySQL and Prisma/Postgres artifacts alongside the MongoDB implementation, creating two competing persistence paths and inconsistent achievement and score behavior. The solution was to remove all MySQL dependencies (mysql2, Prisma schema, SQL seed scripts) from the codebase and ensure that every active runtime path - scores, sessions, achievements, user records - writes to MongoDB through Mongoose models. The backend now has a single coherent persistence architecture.

15.2 Score and Session Synchronization

Some games submit scores naturally, while others submit richer sessions, which created a situation where Spirit Sapling completions appeared in session history but not in score history. The solution was to synchronize the two models in the sessions service: whenever a session is created with a non-null score, the service also creates a ScoreSubmission record and upserts a UserHighScore document in the same call, ensuring consistent visibility across all progress views.

15.3 AI Output Reliability

Gemini can return malformed text, invalid JSON, or - in the Half Moon AI mode - illegal moves such as occupied spaces or cards not in the AI's hand. The solution is a double-validation pattern. The backend constrains the prompt, parses defensively, and returns null on any parse failure. The frontend then validates the returned move against the current game state before accepting it. This makes the AI helpful without making it authoritative over game logic.

15.4 Achievement Recovery

If a session saved successfully but the corresponding badge write failed - for example, due to a transient network error - the Achievements page could show incorrect progress. The achievements service now reconciles badge state from MongoDB history on every catalog request: if an existing GameSession or UserHighScore proves that a badge criterion was met, the service restores the missing UserBadge row transparently.

15.5 Spirit Sapling Layout Responsiveness

Spirit Sapling required repeated layout refinement as the tree visual and guardian rail grew visually dense at stages 5 and 6. At narrow viewports, the guardian rail overflowed vertically, and the fruit basket button rendered outside the game canvas. The solution was to move the basket button into the stage as an absolute-positioned overlay and to size the guardian rail using a viewport-relative grid, keeping all four guardian cards visible at all supported viewport widths.

15.6 Phaser and React Lifecycle Synchronization

Embedding Phaser 3.90 within React 19's component lifecycle requires careful management to prevent multiple game instances from initializing on re-renders, which leads to canvas conflicts

and memory leaks. The solution is to initialize Phaser inside a `useEffect` hook with an empty dependency array so it runs exactly once on mount, and to call `game.destroy(true)` in the cleanup function so the instance is correctly destroyed when the component unmounts. The game container div is accessed via `useRef` to prevent React from managing the DOM node that Phaser owns.

16. Future Improvements

The future vision of Whisperwind Grove extends far beyond its current prototype state. The long-term objective is to evolve the platform into a highly polished, emotionally immersive, and technically reliable interactive ecosystem capable of supporting evolving gameplay experiences, intelligent NPC interaction, live community events, and dynamic progression systems.

One of the highest development priorities is ensuring long-term technical stability across all existing games. Future iterations will focus heavily on eliminating gameplay bugs, improving runtime reliability, refining backend synchronization, and optimizing frontend rendering systems to ensure that each game can operate consistently without crashes, persistence failures, or synchronization issues. This includes stronger testing pipelines, automated validation systems, improved state recovery, and more robust error handling.

Additional sound effects, ambient audio layers, and adaptive musical systems will also be integrated to strengthen immersion and emotional atmosphere. Multiple level variations and difficulty options will be introduced across the platform to improve replayability and long-term player engagement.

16.1 Spirit Drift Expansion

Spirit Drift will evolve into a broader elemental spirit experience featuring multiple playable spirit types with distinct mechanics, visual identities, movement patterns, and abilities. Planned elemental spirits include: wind spirits, water spirits, fire spirits, forest spirits, and celestial spirits.

Each spirit will introduce unique particle systems, movement behaviors, environmental effects, and achievement paths. These additions are intended to deepen gameplay variety while preserving the game's calming artistic identity.

16.2 Delivery on the Wind Expansion

Delivery on the Wind will be expanded into a more socially interactive world. The current gameplay loop will evolve through the addition of: village NPCs, townfolk interactions, dialogue systems, side quests, delivery requests, and environmental storytelling.

Players controlling Kiki will be able to communicate with townspeople, build relationships, unlock narrative events, and participate in emotionally driven interactions throughout the town.

An additional obstacle-based mini-game inspired by side-scrolling flight mechanics will also be integrated into Delivery on the Wind. This gameplay mode will introduce airborne obstacle

navigation sequences requiring timing, reflexes, and precision movement while maintaining the platform's whimsical visual atmosphere.

17.3 Spirit Sapling Expansion

Spirit Sapling will continue evolving into a more emotionally intelligent and personalized experience.

Planned improvements include: advanced sapling interaction systems, expanded growth-stage animations, dynamic environmental reactions, enhanced fruit evolution systems, and richer guardian dialogue.

Future versions will also integrate Character AI-based systems, enabling players to maintain persistent conversations with guardians over time. Players may eventually be able to: create their own guardians, customize personalities, train conversational behaviors, and develop emotionally persistent AI companions.

This system aims to transform guardians from scripted assistants into evolving narrative entities capable of supporting long-term emotional attachment.

17.4 Rise of the Half Moon Expansion

Rise of the Half Moon will receive expanded map designs, environmental diversity, and improved strategic gameplay systems.

Planned additions include: multiple map layouts, monthly moon recap environments, improved progression balancing, and more advanced score mechanics.

Future UX improvements will focus on smoother transitions, improved readability, more cinematic visual feedback, and stronger gameplay clarity.

17.5 Platform-Wide Improvements

The profile and settings systems will also be redesigned to support deeper personalization and accessibility.

Planned additions include: customizable player profiles, guardian collections, audio configuration systems, accessibility settings, save preferences, and expanded progression analytics.

To strengthen long-term engagement, the platform will introduce: live challenges, seasonal events, rotating achievements, limited-time progression goals, and community-based objectives.

A significantly expanded achievement ecosystem is planned to encourage exploration, experimentation, mastery, and emotional interaction.

17.6 Security and Infrastructure Improvements

As the platform continues to grow, stronger security systems will become increasingly important.

Future backend improvements will include: secure authentication systems, improved API protection, rate limiting, encrypted session handling, database validation layers, anti-cheat protections, and improved cloud infrastructure.

These improvements aim to support a scalable and secure live-service architecture capable of supporting long-term expansion.

Ultimately, the future vision of Whisperwind Grove is not simply to become a collection of games, but rather an emotionally immersive digital world where technology, storytelling, art, and intelligent interaction coexist harmoniously.

17. Conclusion

Whisperwind Grove demonstrates how a browser-based game platform can combine lightweight mini-games with serious full-stack engineering. The platform integrates a typed React 19 and Vite 7 frontend, Phaser 3.90 game scenes, an Express 5 API, MongoDB persistence through Mongoose 9, JWT authentication, leaderboard queries, persistent session history, achievement systems, Gemini 2.0 Flash gameplay interactions, and ElevenLabs voice synthesis into a unified production-oriented TypeScript monorepo.

The most important architectural feature of the project is that each game remains creatively distinct while still participating in one shared progression ecosystem. Spirit Drift contributes arcade-style score competition and reaction-focused gameplay. Delivery on the Wind contributes timed navigation, optimization mechanics, and movement-based interaction. Spirit Sapling contributes emotionally driven AI communication and long-form nurturing systems. Rise of the Half Moon contributes strategic card gameplay supported by an AI opponent. MongoDB persistence binds these four experiences together through shared achievements, score histories, progression milestones, and gameplay sessions.

One of the most significant engineering contributions of the project is the intentional bounding of artificial intelligence systems. Gemini 2.0 Flash provides constrained prompts; its outputs are parsed defensively, and gameplay actions remain validated through deterministic game logic before being accepted by the system. This principle. AI as a bounded contributor rather than an unrestricted authority ensures that the games remain fair, stable, predictable, and testable regardless of model behavior.

At the same time, Whisperwind Grove was never intended to function solely as a technical exercise. A substantial portion of the development process focused on artistic direction, environmental atmosphere, aesthetic consistency, animation refinement, guardian identity design, soundtrack curation, visual storytelling, and emotional tone. Considerable artistic liberty was intentionally taken throughout development in order to preserve the project's identity as an emotionally immersive interactive world rather than a purely utilitarian software system.

The visual language of the platform was carefully curated to evoke calmness, wonder, nostalgia, and emotional warmth. Color palettes, environmental compositions, guardian personalities, interface transitions, and soundtrack choices were all iteratively refined to support a cohesive artistic experience. Many development decisions prioritized emotional atmosphere and aesthetic harmony alongside engineering efficiency. It is a coherent software ecosystem demonstrating frontend architecture, backend service design, database modeling, AI integration, game-engine integration, achievement persistence, voice synthesis systems, and iterative UX refinement, all while preserving a strong artistic identity centered around empathy, imagination, and emotionally meaningful interaction.

Ultimately, the project demonstrates that modern software engineering and artistic expression are not opposing disciplines. Through Whisperwind Grove, this work argues that interactive systems can simultaneously function as technical achievements, emotional experiences, and artistic creations.

References

- Claypool, M., & Claypool, K. (2006). Latency and player actions in online games. *Communications of the ACM*, 49(11), 40–45.
- Deci, E. L., & Ryan, R. M. (1985). *Intrinsic motivation and self-determination in human behavior*. Plenum Press.
- Deterding, S., Dixon, D., Khaled, R., & Nacke, L. (2011). From game design elements to gamefulness: Defining gamification. *MindTrek 2011*, 9–15.
- ElevenLabs. (2024). ElevenLabs TTS API documentation. <https://elevenlabs.io/docs>
- Phaser. (2024). Phaser 3 documentation and API reference. <https://phaser.io/phaser3>
- Google DeepMind. (2024). Gemini API developer documentation. <https://ai.google.dev/docs>
- Hamari, J., Koivisto, J., & Sarsa, H. (2014). Does gamification work? A literature review of empirical studies. *HICSS 2014*, 3025–3034.
- Lazzaro, N. (2004). *Why we play games: Four keys to more emotion without story*. GDC 2004.
- MongoDB. (2024). MongoDB documentation: Aggregation pipeline. <https://www.mongodb.com/docs>
- Nacke, L. E., & Grimshaw, M. N. (2011). *Player-game interaction through affective sound. In-game sound technology and player interaction*. IGI Global.
- Norman, D. A. (2004). *Emotional design: Why we love (or hate) everyday things*. Basic Books.
- Pinelle, D., Wong, N., & Stach, T. (2008). *Heuristic evaluation for games: Usability principles for video game design*. CHI 2008.
- React. (2024). React 19 documentation. <https://react.dev>
- Russoniello, C. V., O'Brien, K., & Parks, J. M. (2009). The effectiveness of casual video games in improving mood and decreasing stress. *Journal of CyberTherapy and Rehabilitation*, 2(1), 53–66.
- Ryan, R. M., Rigby, C. S., & Przybylski, A. (2006). The motivational pull of video games: A self-determination theory approach. *Motivation and Emotion*, 30, 347–363.
- Vite. (2024). Vite 7 documentation. <https://vitejs.dev/guide/>